

Indexation Web

Janvier 2026

Lara Perinetti - Team Lead Search



Ask Le Chat anything



∞ Research



Think



Tools



Parlez-moi de vous

- Combien vous êtes dans chaque groupe ?
- Quels langages de programmation vous connaissez ?
- A quel point vous êtes familiers avec les notions du web ?
 - HTML, css, javascript ?
 - Flux rss ?
 - robots.txt ?
- Avez-vous déjà étudié le NLP ? IR ?

Cours Indexation Web

- 6h CM en commun
- 3 * 3h TP
 - Crawler
 - Index
 - Moteur de recherche final

Cours Indexation Web

Rendus des TPs

- Les TPs seront à rendre dans un **repository git public**
- A chaque fin de TP, il faudra push votre avancement
- Vous aurez une semaine de plus si vous n'avez pas fini pour me (re) envoyer le code amélioré et m'envoyer la version définitive
- TP Crawler - 13/01
- TP Index - 13/01
- TP Moteur - 14/01
- Tous les TPs sont à envoyer au plus tard le **21/01**

Introduction

Partie 1: Constitution d'un index web

1. Architecture d'un moteur de recherche
2. Crawler le web
3. Document Processing

Partie 2: Traitement de la requête et ordonnancement des résultats

1. Index
2. Compréhension de la requête
3. Ordonnancement des résultats et évaluation
4. Retrieval Augmented Generation (RAG)

Qu'est-ce qu'un moteur de recherche?

Differents types

- **Web Search:** Google, Qwant, DuckDuckGo, Ecosia etc.
 - Differents formats: HTML, pdf, docx etc.
 - Champs les plus communs: title, content, page rank
- **Shopping Search:** Amazon, Cdiscount, La Redoute etc.
 - Leurs propres structures de données, appliquer des filtres
 - Champs les plus communs: nom du produit, prix, image
- **Desktop Search:** sur les ordinateurs personnels
 - fichiers, emails, pages webs dans l'historiques des recherches
 - Séparer les sources de données
- **IA Search:**
 - Le contenu sémantique est bien plus important pour que le modèle génère une réponse avec
 - On cherche à avoir des sources fiables et de qualité

Qu'est-ce qu'un moteur de recherche?

SERP
Search
Engine
Results
Pages

Paid
Results

Organic
Results

Search
Engine
Features

Qwant

canal +

×

🔍

🔍 All

📰 News

🖼️ Images

📺 Videos

🛒 Shopping

📍 Maps

France ▾ Any freshness ▾

+

boutique.canalplus.com › Série_Limitée › Canal_Plus

Série Limitée - CANAL+, DISNEY+ & PARAMOUNT+

Ad Série Limitée **CANAL+** & DISNEY+ & PARAMOUNT+pour 25,99€/mois pdt 12 mois puis 35,99€/mois. A Noël, partagez vos films préférés avec la Série Limitée **CANAL+**,...
CANAL+ Série · Offre CANAL+ -26 ans · CANAL+ Ciné Série · CANAL+ Sport

Ads by Microsoft ⓘ

+

canalplus.com

myCANAL : tv, sports, séries, films en streaming en direct live ou...

Regarder les meilleurs programmes, films, séries, sports en streaming direct ou en replay.
Le programme TV de toutes les chaines est gratuit et sans pub.

W

fr.wikipedia.org › wiki › MyCanal

MyCanal

My**Canal**, stylisé en my**CANAL**, est un service français de distribution de contenu par Internet du groupe **Canal+** lancé en décembre 2013. Il permet d'accéder aux programme...

Canal+

Chaîne de télévision française

Canal+ est une chaîne de télévision généraliste nationale française privée à péage, axée sur le cinéma et le sport. Lancée le 4 novembre 1984 par Havas, elle fut la toute première chaîne privée à péage en France. Elle appartient à la Société...

Read more on Wikipedia

Pays : France

Site Web : www.canalplus.com

Propriétaire : Vivendi

Wikipedia.org · How to contribute?

Share your feedback

8

SERP
Search
Engine
Results
Pages

Paid Results

Organic Results

Search Engine Features

Qwant canal +

🔍 All 📰 News 🖼️ Images ▶ Videos 🛒 Shopping 📍 Maps

France ▾ Any freshness ▾

+ boutique.canalplus.com › Série_Limitée › Canal_Plus

Série Limitée - CANAL+, DISNEY+ & PARAMOUNT+

Ad Série Limitée **CANAL+** & DISNEY+ & PARAMOUNT+ pour 25,99€/mois pdt 12 mois puis 35,99€/mois. A Noël, partagez vos films préférés avec la Série Limitée **CANAL+**,...

[CANAL+ Série](#) · [Offre CANAL+ -26 ans](#) · [CANAL+ Ciné Série](#) · [CANAL+ Sport](#)

Ads by Microsoft ⓘ

+ canalplus.com

myCANAL : tv, sports, séries, films en streaming en direct live ou...

Regarder les meilleurs programmes, films, séries, sports en streaming direct ou en replay. Le programme TV de toutes les chaines est gratuit et sans pub.

fr.wikipedia.org › wiki › MyCanal

MyCanal

My**Canal**, stylisé en my**CANAL**, est un service français de distribution de contenu par Internet du groupe **Canal+** lancé en décembre 2013. Il permet d'accéder aux programme...

Canal+

Chaîne de télévision française

Canal+ est une chaîne de télévision généraliste nationale française privée à péage, axée sur le cinéma et le sport. Lancée le 4 novembre 1984 par Havas, elle fut la toute première chaîne privée à péage en France. Elle appartient à la Société...

[Read more on Wikipedia](#)

Pays : France

Site Web : www.canalplus.com

Propriétaire : Vivendi

Wikipedia.org · [How to contribute?](#)

[Share your feedback](#)

Information Retrieval

- *“Information retrieval is a field concerned with the structure, analysis, organization, storage, searching, and retrieval of information.”* (Salton, 1968)
- Définition moderne : La recherche d'information est la science qui permet d'identifier et d'extraire des documents pertinents à partir d'une vaste collection de données non structurées, en réponse à un besoin d'information spécifique.
- Exclus : Systèmes de recommandation (Netflix, Spotify)
- Défis fondamentaux :
 - Subjectivité de la pertinence
 - Inadéquation requête-besoin

Information Retrieval

Qu'est-ce qu'un document web?

- On va se concentrer sur les pages HTML
Balises les plus courantes:

Exemple

- Structure et nom des balises :
 - Les balises suivent des normes réglementées (W3C).
 - Elles peuvent contenir : Texte / Images / Vidéos / Script / Liens
 - Métadonnées :
 - Les métadonnées (dans la balise <head>) sont des informations sur le document et non sur son contenu.
 - Exemple : type de document, description, mots-clés pour les moteurs de recherche.

```
<head>
  <title>Mon site web</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Titre principal</h1>
  <div>
    <p>Exemple de paragraphe</p>
  </div>
</body>
```

Information Retrieval

Qu'est-ce qu'une requête web ?

- Spécificités d'une requête web: souvent courte (un ou deux termes) et ambiguë
- On définit **3 intentions principales** pour une requête web
 - Navigationnelle
 - Transactionnelle
 - Informationnelle
- On peut définir **plusieurs topics** pour une requête web
 - Exemple:
requête: Jaguar
topics: animal / voiture

Information Retrieval

What is a web query ?

Exemples de requetes d'utilisateurs Qwant en décembre 2022

youtube

[https://
www.ensai.fr/](https://www.ensai.fr/)

cabinet comptable
cfac cenon

Nice Marseille
train

maladroit en
anglais

le monde

Voiture occasion

Reine
d'angleterre

météo paris

Atelier solidaire à
Dignes les bains

Jean luc godard

Argentine france

Avatar 2

Ikea

Les classer par intention (informationnelle, transactionnelle, navigationnelle)

Information Retrieval

What is a web query ?

Exemples de requetes d'utilisateurs Qwant en décembre 2022

youtube

maladroit en
anglais

Reine
d'angleterre

Argentine france

https://
www.ensai.fr/

cabinet comptable
cfac cenon

météo paris

Avatar 2

Nice Marseille
train

Voiture occasion

le monde

Atelier solidaire à
Dignes les bains

Jean luc godard

Ikea

Les classer par intention (informationnelle, transactionnelle, navigationnelle)

Information Retrieval

What is a web query ?

Exemples de requetes d'utilisateurs Qwant en décembre 2022

youtube

maladroit en
anglais

Reine
d'angleterre

Argentine france

météo paris

[https://
www.ensai.fr/](https://www.ensai.fr/)

cabinet comptable
cfac cenon

le monde

Atelier solidaire à
Dignes les bains

Avatar 2

Nice Marseille
train

Voiture occasion

Jean luc godard

Ikea

Les classer par topics (pas plus de deux topics par requête)

Information Retrieval

What is a web query ?

Exemples de requetes d'utilisateurs Qwant en décembre 2022

youtube

maladroit en
anglais

Reine
d'angleterre

Argentine france

[https://
www.ensai.fr/](https://www.ensai.fr/)

cabinet comptable
cfac cenon

météo paris

Avatar 2

Nice Marseille
train

le monde

Atelier solidaire à
Dignes les bains

Voiture occasion

Jean luc godard

Ikea

shopping, news, entertainment, sports, etc.

Partie 1

Constitution d'un index web

Architecture d'un moteur de recherche

Architecture d'un moteur de recherche

Objectifs

1. Qualité

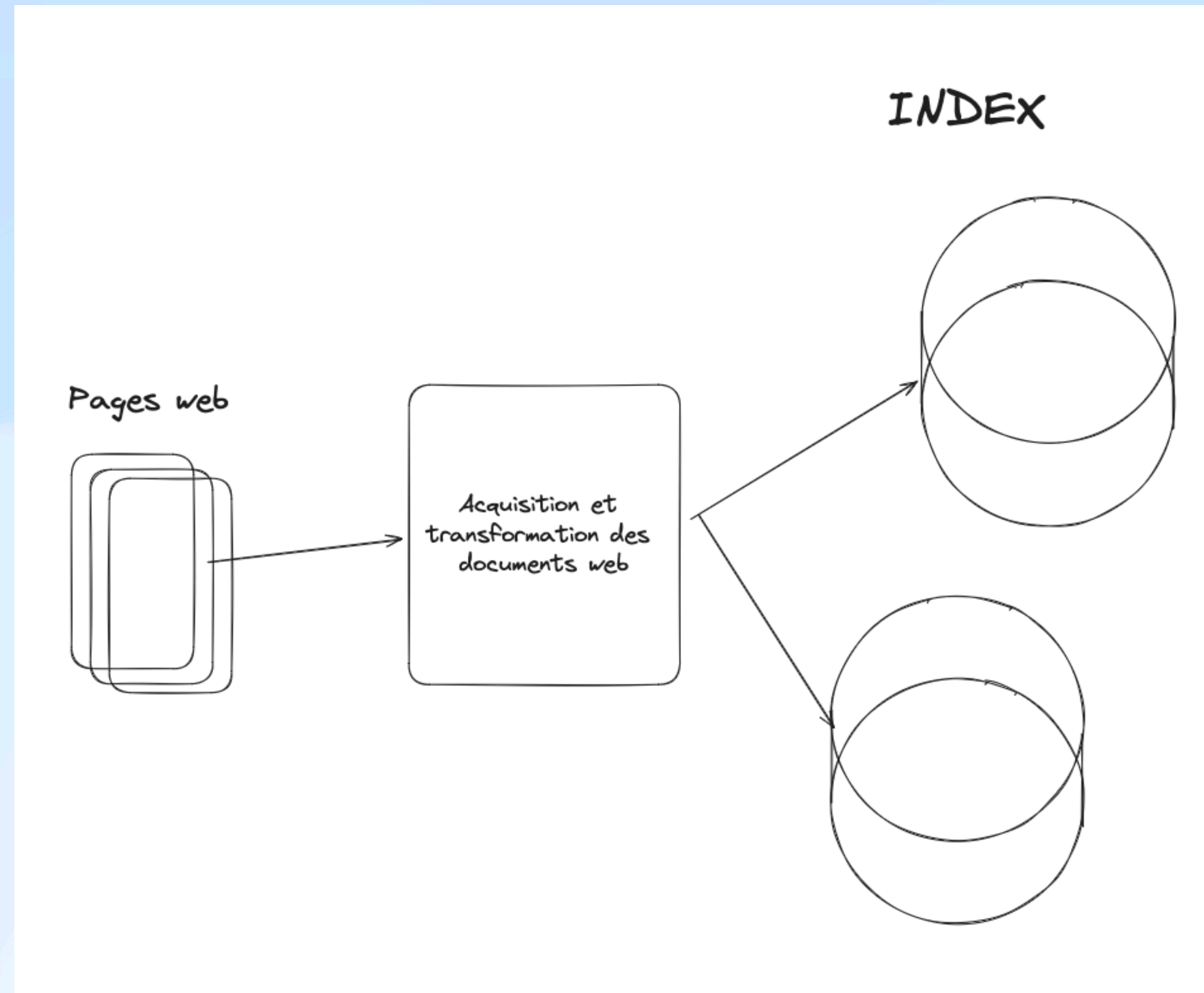
- Objectif: Assurer la récupération des **documents les plus pertinents** en réponse à une requête utilisateur.
- Dans le but de: Répondre avec précision aux attentes et besoins des utilisateurs.

2. Rapidité

- Objectif: Traiter les requêtes des utilisateurs **le plus rapidement possible**.

Architecture d'un moteur de recherche

Des pages webs à l'index



Des pages webs à l'index

Text Acquisition

- Identifie et rend disponible les documents que l'on voudra montrer à l'utilisateur

1. Quels documents ?

- partir d'une collection de documents clés en main
- chercher ces documents via une tâche de **crawl** ou de **scrap**

2. Où les stocker?

Une base de données relationnelle peut être utilisée pour stocker les documents et leurs métadonnées.

Des pages webs à l'index

Text Acquisition

1. La notion de **crawler**

- Un crawler est conçu pour **suivre les liens** sur les pages web pour **découvrir** et télécharger de nouvelles pages
- Un bon crawler doit être capable de gérer efficacement l'énorme volume de nouvelles pages sur le Web, tout en veillant à ce que les pages qui ont pu être modifiées depuis la dernière visite d'un crawler restent "fraîches" pour le moteur de recherche

2. La notion de **scraper**

- Un scraper est un programme qui **extraît des données** spécifiques d'**une page web**. Contrairement au crawler, son objectif principal est de récupérer des informations ciblées sur une ou plusieurs pages web.

Des pages webs à l'index

Text Acquisition - Crawler

- Utilisation:
 - Le crawler est un des composants principaux d'un moteur de recherche
 - Archivage du web (Internet Archive, Common Crawl)
 - Exploration + analyses de données sur le web (ex: Attributor)
- Challenges:
 - **Scalabilité** —> Le web est très vaste et en constante évolution.
 - **Selection** des documents
 - Obligations de **politesse**
 - Qualité et **malveillance** des sites web

Des pages webs à l'index

Text Transformation

- Link extraction / analysis
- Information extraction (language, topic etc.)
- Parsing —> hierarchy extraction
- Stopping —> removing stop words
- Stemming / Lemmatization

Des pages webs à l'index

Création d'index

- **Entrée** : Utiliser la sortie de la transformation de texte pour créer des index.
- Objectif : Les index contiennent les informations nécessaires sur les documents pour permettre une recherche rapide et efficace.
- Contraintes :
 - **L'indexation** doit être **rapide** pour gérer un grand nombre de documents.
 - L'algorithme doit être optimisé en temps et en espace pour garantir une **utilisation efficace des ressources**.

Architecture d'un moteur de recherche

De la requête aux résultats finaux

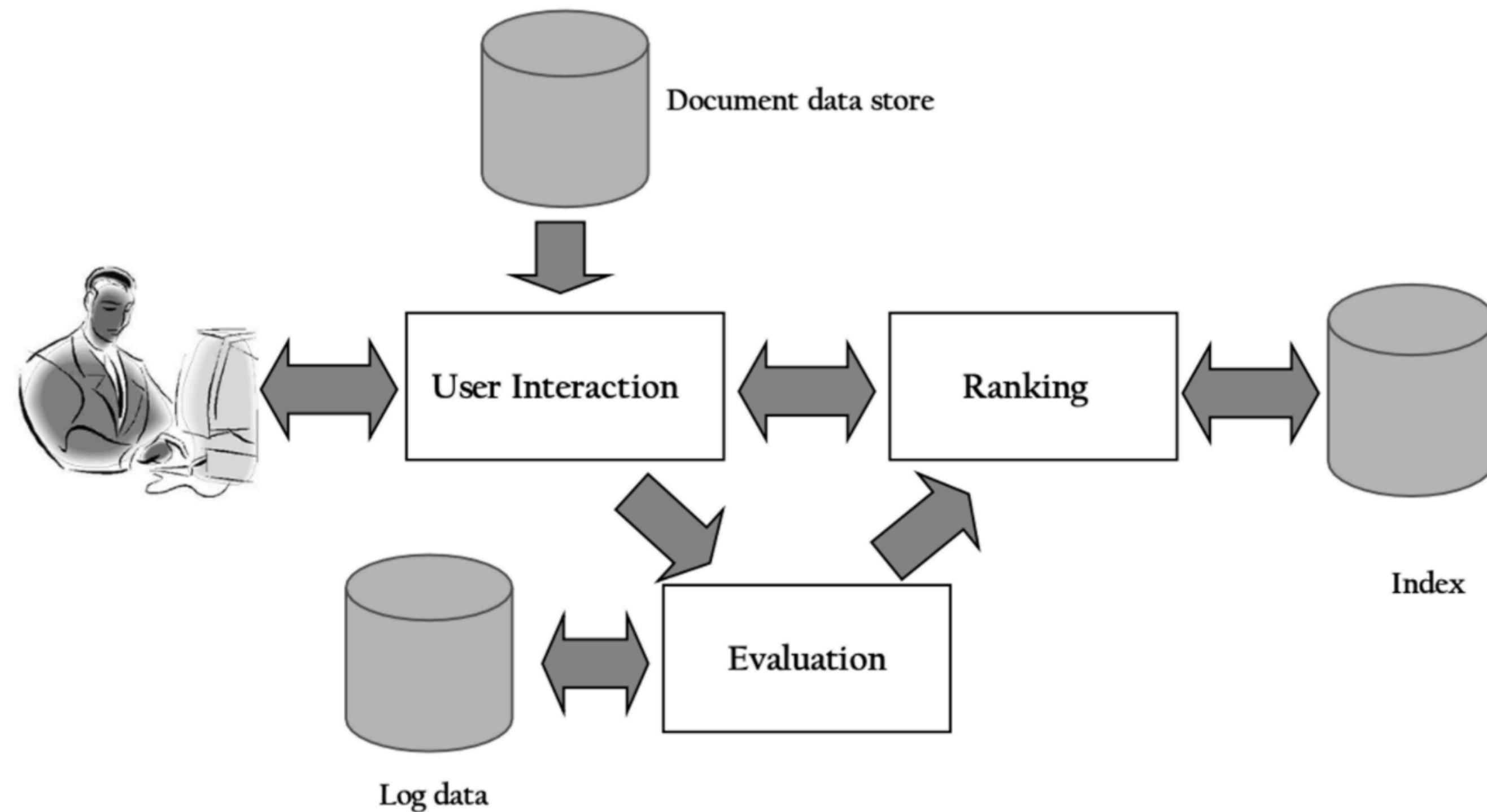


Fig. 2.2. The query process

De la requête aux résultats finaux

Interaction utilisateur

1. Réception de la requête

- Le système reçoit en entrée :
 - **Texte** de la requête saisi par l'utilisateur
 - **Métadonnées** associées:
 - Langue de l'utilisateur.
 - Topics ou thématiques liés à la requête.
 - Informations contextuelles (heure, jour de la semaine, etc.).

2. Étapes de traitement

- Transformation de la requête pour qu'elle soit compatible avec l'index du système :
 - **Correction orthographique** (Spell checking).
 - **Enrichissement** ou augmentation de la requête :
 - Ajout de synonymes.
 - Expansion sémantique (ex. : “cinéma” → “films”, “projections”).
 - Prise en compte des métadonnées pour ajuster les résultats.

De la requête aux résultats finaux

Ranking

1. Input : Documents filtrés

- Le système reçoit les documents qui ont passé le filtre initial en fonction de la requête utilisateur.

2. Attribution d'un score aux documents

- Chaque document est analysé et évalué par rapport à la requête.
- Le score reflète la pertinence du document pour répondre à la demande de l'utilisateur.

3. Objectifs clés

- **Efficacité** : Traiter un grand nombre de requêtes rapidement (temps de réponse minimal).
- **Effectivité** :
 - Garantir un classement de haute qualité pour maximiser la pertinence des résultats.
 - Assurer que les documents les plus utiles apparaissent en priorité.

De la requête aux résultats finaux

Evaluation

- **Évaluation Offline**

- Définition : Tester le moteur de recherche sur des données préenregistrées.
- Objectif : Mesurer la qualité intrinsèque des algorithmes.
- Indicateurs clés :
 - Précision (Precision) / Rappel (Recall)
 - Mean Average Precision (MAP)
 - NDCG (Normalized Discounted Cumulative Gain)
- Avantage : Rapide et reproductible.
- Limite : Ne prend pas en compte le comportement réel des utilisateurs. Evaluation “online”

De la requête aux résultats finaux

Evaluation

- **Évaluation Online**

- Définition : Tester le moteur directement sur des interactions réelles avec les utilisateurs.
- Objectif : Évaluer la performance en situation réelle.
- Méthodes clés :
 - A/B Testing : Comparer deux versions du moteur.
 - CTR (Click-Through Rate) : Taux de clics sur les résultats.
 - Dwell Time : Temps passé par l'utilisateur sur une page.
- Avantage : Donne des insights basés sur l'expérience utilisateur.
- Limite : Plus complexe et nécessite des données utilisateur.

Partie 1

Constitution d'un

index web

Crawler le web

Crawler le web

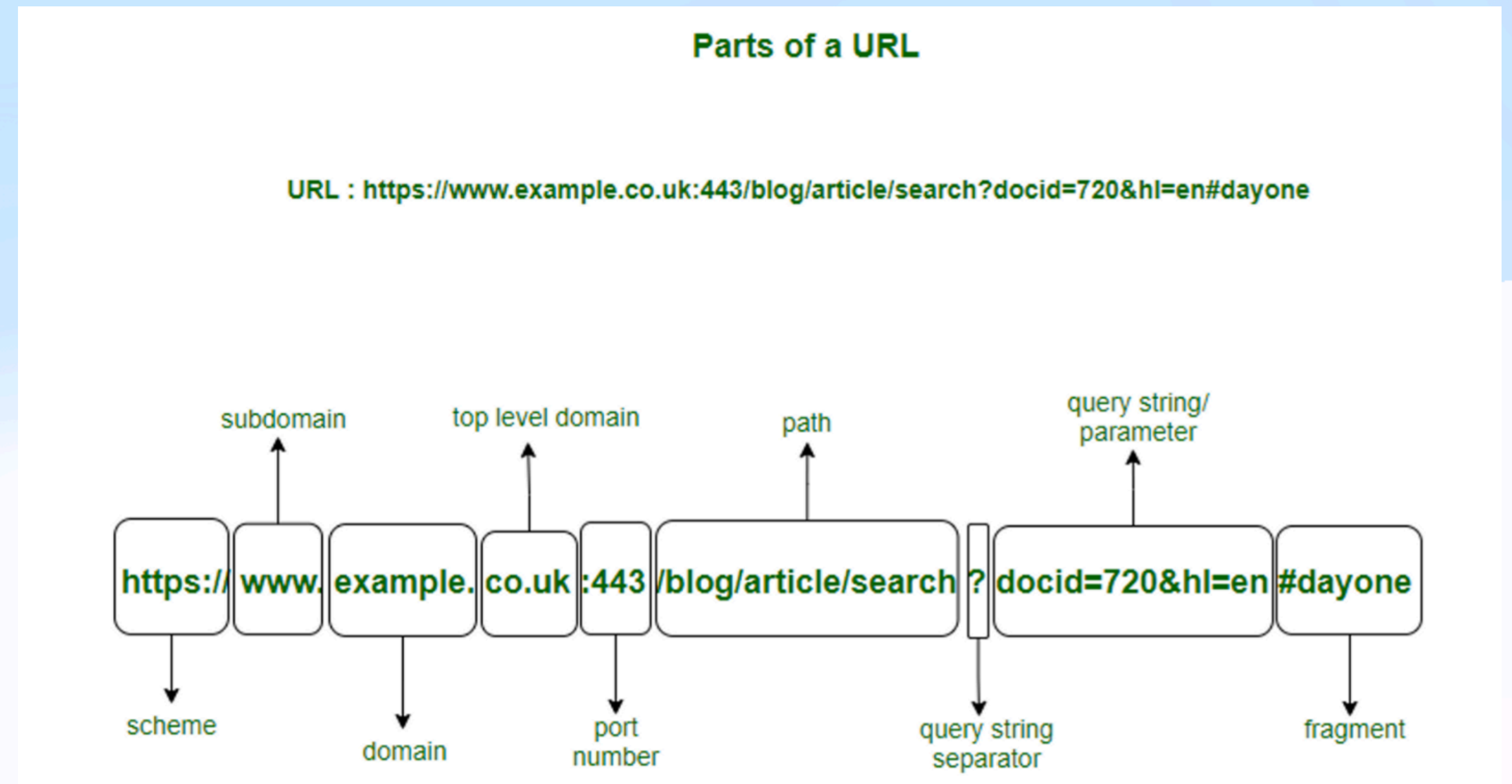
Décider de ce qu'il faut rechercher

- Les défis d'un crawler web
 - Un web immense et en constante évolution
 - **Croissance rapide** : Le web contient des milliards de pages et continue de s'étendre.
 - **Changements fréquents** :
 - Les pages web sont mises à jour régulièrement.
 - Les documents utiles aujourd'hui peuvent perdre de leur pertinence avec le temps.
 - Contraintes de **stockage**
 - Impossibilité de tout stocker
 - Priorisation des contenus
- Évaluation des crawlers
 - **Couverture** (Coverage) : Mesure la quantité de documents pertinents récupérés.
 - **Fraîcheur** (Freshness) : Indique à quel point les pages stockées sont actualisées.
- Règles de **politesse** pour les crawlers
 - Respect des serveurs web : Espacer les visites pour une URL donnée afin d'éviter une surcharge.
 - Suivre les directives du fichier **robots.txt** des sites web.

Crawler le web

Qu'est-ce qu'une page web ?

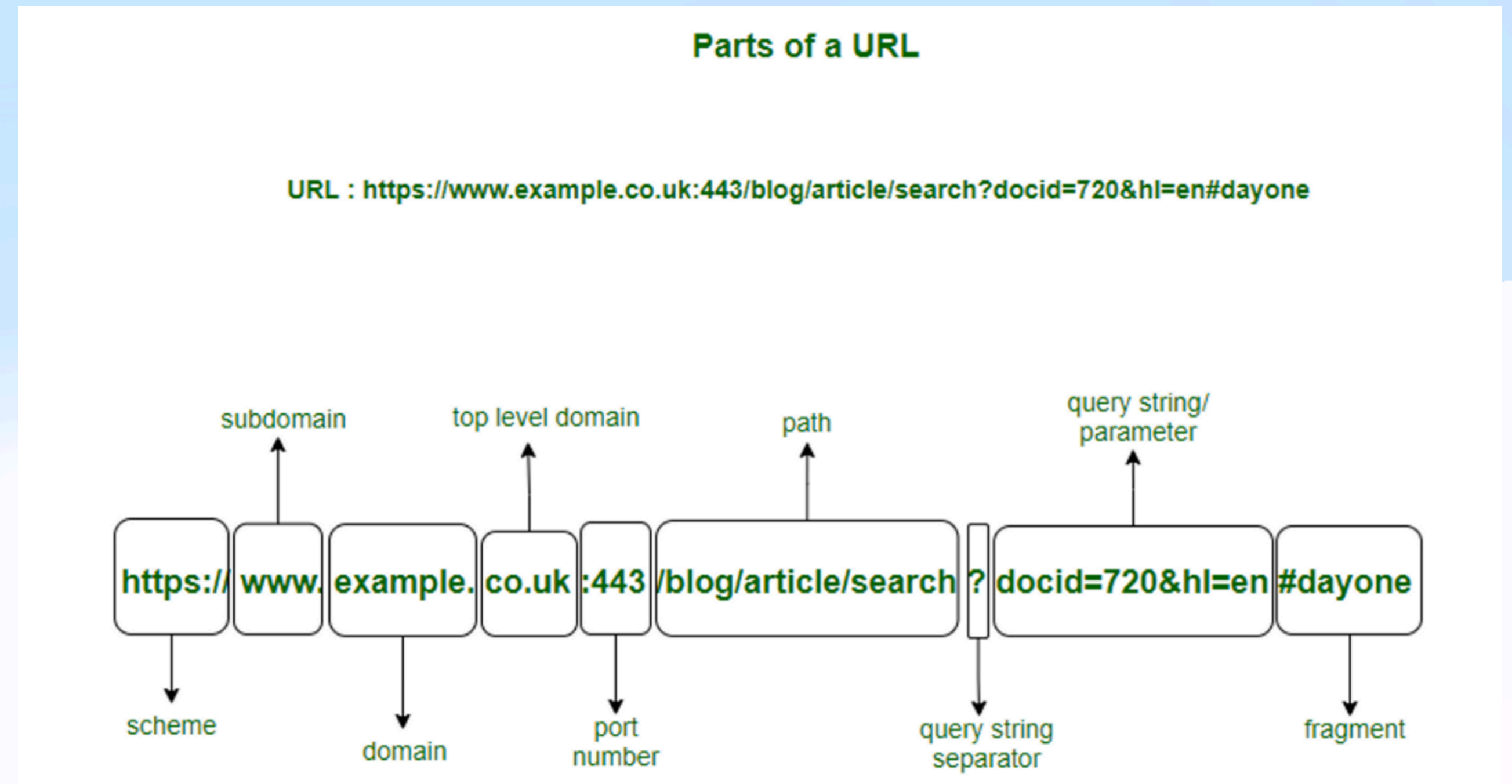
- Scheme
http | https | ftp | smtp
- Subdomain
indique le type de ressource
www | blog | audio etc.
- Domain
indique l'organisation
- TLD
indique le type d'organisation à laquelle le site web est enregistré.
pays: fr | co.uk | de
region: re
autre: com | org | net



Crawler le web

Qu'est-ce qu'une page web ?

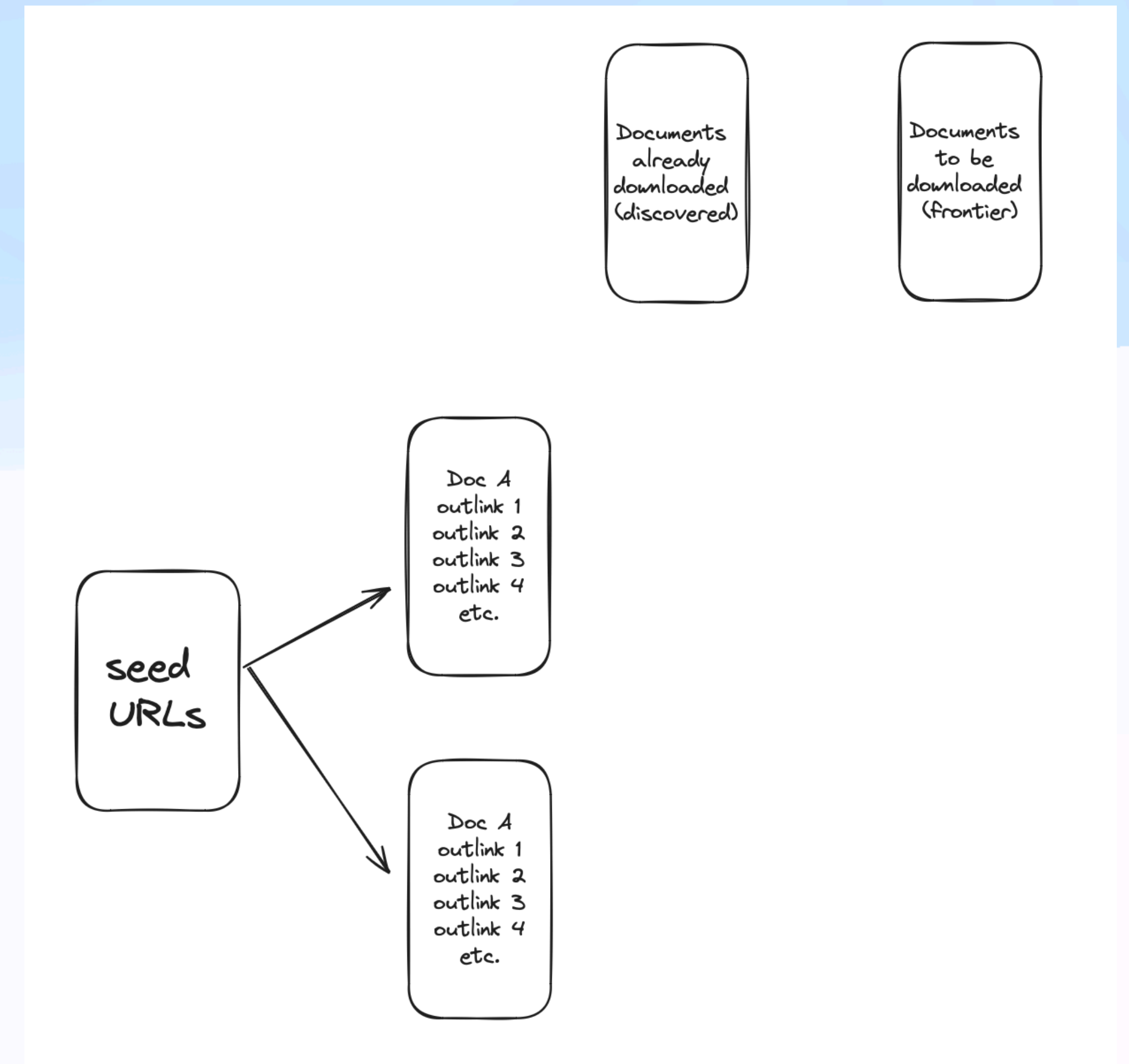
- Path
indique la localisation de la page dans son domaine
- Query
les paramètres d'un site web.
- Fragment
pour arriver directement à une certaine balise dans le document html



Crawler le web

Fonctionnement d'un crawler (Algorithme)

1. Entrée : Une liste initiale d'URLs (seed URLs).
2. Ajout à la file d'attente (Frontier)
 - Les URLs sont ajoutées à une file d'attente qui gère les requêtes :
 - La file d'attente peut être :
 - FIFO standard (ordre chronologique).
 - Prioritaire pour privilégier les pages importantes.
3. Récupération des pages web
4. Analyse des pages téléchargées
 - Identifier les balises de liens (<a>) contenant d'autres URLs.
 - Pour trouver de nouvelles URLs utiles à explorer
5. Ajout des nouvelles URLs
 - Si une URL n'a pas encore été visitée, elle est ajoutée à la frontière.
6. Cycle continu jusqu'à arrêt jusqu'à ce que :
 - La frontière soit vide (plus aucune URL à visiter).
 - Les ressources soient épuisées (disque plein, quotas atteints).



Crawler le web

Controler le crawler: robots.txt

- Les propriétaires de sites Web utilisent le fichier /robots.txt pour donner des instructions sur leur site aux crawlers
- Exemples de robots.txt
 - Site important
 - Site de niche
- Guidelines de google
- Informations sur le crawler de Qwant

Crawler le web

La notion de fraîcheur

1. Contraintes liées à la **fraîcheur**

- **Impossibilité de tout vérifier** : Il est irréaliste de contrôler constamment la fraîcheur de toutes les pages du web.

2. Priorisation des vérifications

- Pages **importantes** : Les pages avec un contenu critique ou très consulté sont vérifiées en priorité.
- Pages **fréquemment mises à jour** : Les pages qui changent souvent sont surveillées plus régulièrement.

3. Mesure de la fraîcheur

- Définition : La fraîcheur est la proportion des pages explorées qui contiennent des informations actuelles.
- Challenge : Si une page est mise à jour toutes les 2 secondes, maintenir sa fraîcheur devient extrêmement coûteux en ressources.

Crawler le web

La notion de fraîcheur

- Le cycle de vie d'une page :
 1. Lorsqu'une **page est crawlée**, elle est immédiatement considérée comme **fraîche**.
 2. Dès qu'un **changement** est détecté sur la page, celle-ci devient **périmée**.
 3. Une page a un **âge** de 0 jusqu'à sa première modification, puis cet âge **augmente** jusqu'à sa prochaine exploration.

Crawler le web

La notion de fraîcheur

- Jour 1 : Première exploration
 - Toutes les pages sont explorées pour la première fois, leur **âge est de 0 jours** (elles viennent d'être explorées).
 - Toutes les pages sont considérées comme fraîches car elles viennent d'être explorées et sont à jour.
 - **Fraîcheur du site = 100% (toutes les pages sont fraîches).**
- Jour 3 : Mise à jour partielle, seules les pages 1 et 2 sont redécouvertes
 - Pages 1 et 2 sont modifiées (nouveau contenu ajouté). -> Les pages 1 et 2 restent fraîches, car elles viennent d'être explorées.
 - Pages 3, 4 et 5 n'ont pas changé
 - Les pages 3, 4 et 5 sont maintenant considérées comme périmées car leur contenu n'a pas été mis à jour depuis la première exploration.
 - **L'âge des pages 3, 4 et 5 devient de 2 jours** (elles n'ont pas été explorées depuis 2 jours).
 - **Fraîcheur du site = 40% (2 pages fraîches sur 5).**
- Jour 5 : Nouvelle exploration
 - Toutes les pages sont explorées de nouveau.
 - Pages 1 et 2 sont explorées à nouveau, leur **âge est remis à 0 jours** car elles sont mises à jour.
 - Pages 3, 4 et 5 sont également explorées et mises à jour, leur âge est **également remis à 0 jours**.
 - Toutes les pages sont maintenant fraîches à nouveau, car elles ont été explorées récemment.
 - **Fraîcheur du site = 100% (toutes les pages sont fraîches).**

Crawler le web

Sitemaps

- Certains sites web exposent un sitemap aux crawler dans le robots.txt
- Exemple

⊗ Quel est l'intérêt de créer cette sitemap pour l'administrateur du site?

Crawler le web

Sitemaps

- Certains sites web exposent une **sitemap** aux crawler dans le robots.txt
- Exemple

⊗ Quel est l'intérêt de créer cette sitemap pour l'administrateur du site?

- Il indique aux moteurs de recherche des pages qu'ils ne trouveraient pas autrement.
- Cela permet de réduire le nombre de requêtes que le crawler envoie à un site web sans sacrifier la fraîcheur des pages.

Crawler le web

Stocker les documents

- La recherche de documents est couteuse en termes de CPU et de réseau.
- On va vouloir conserver une copie des documents déjà crawlés au lieu d'essayer de les récupérer à chaque fois qu'on lance un tour de crawl.
- En général on utilise une BDD relationnelle avec pour identifiant unique du document un hash de l'URL
- Exemple de BDD: Big Table

Crawler le web

Détecter les documents dupliqués

- Tâche plutôt simple
- On va souvent utiliser des techniques qui calculent les bytes des documents
- Baseline: **checksum technique**:
T r o p i c a l f i s h
54 72 6F 70 69 63 61 6C 20 66 69 73 68 result = 508
- **cyclic redundancy check**, une autre technique qui prend en compte la position des bytes.
- Attention: <http://monsite.com> != <https://monsite.com>

Crawler le web

Détecter les documents “presque” dupliqués (ou near duplicates)

- Tâche plus complexe
- Pour une url donnée, on va chercher à trouver tous ses “presque” duplicats dans la BDD.
- Pour cela, il faut représenter les documents
 - Soit en le hashant (e.g: sim-hash)
 - Soit avec des embeddings (vecteur dense de représentation d'un document)

Crawler le web

Focused crawler

- Dans certains cas, on va vouloir **prioriser certaines pages web** à crawler
- On considère alors
 1. l'importance d'une page ou d'un site par rapport aux autres
 2. la pertinence d'une page ou d'un site par rapport à l'objectif poursuivi par le crawl
 3. la dynamicité, ou la manière dont le contenu d'une page/d'un site a tendance à changer au fil du temps.

Partie 1

Constitution d'un index web

Extraction d'information à partir de documents

Extraire des informations des documents

- Avant d'ajouter nos documents dans un index, on va vouloir en extraire le plus d'informations possibles qui nous aideront à répondre de la manière la plus pertinente
 - Text Processing
 - Structure du document
 - Analyse des liens
 - Document embeddings

Différentes manières d'extraire des informations des sites web

Différentes manières d'extraire des informations

- HTML
 - Les sites de documentation (comme Python docs)
 - Beaucoup de sites gouvernementaux
- Javascript
 - LinkedIn / Airbnb / X etc.
- Flux RSS
 - News
- Dump réguliers
 - Reddit / Wikipedia
- API payantes
 - YouTube / Spotify / Wikipedia

Différents problèmes rencontrés

- HTML
 - Temps de réponse variables qui nécessitent des timeouts adaptés
 - Structure parfois peu sémantique ou mal structurée
- Javascript
 - Besoin d'un navigateur headless
 - Consommation importante de ressources (CPU/mémoire)

Différents problèmes rencontrés

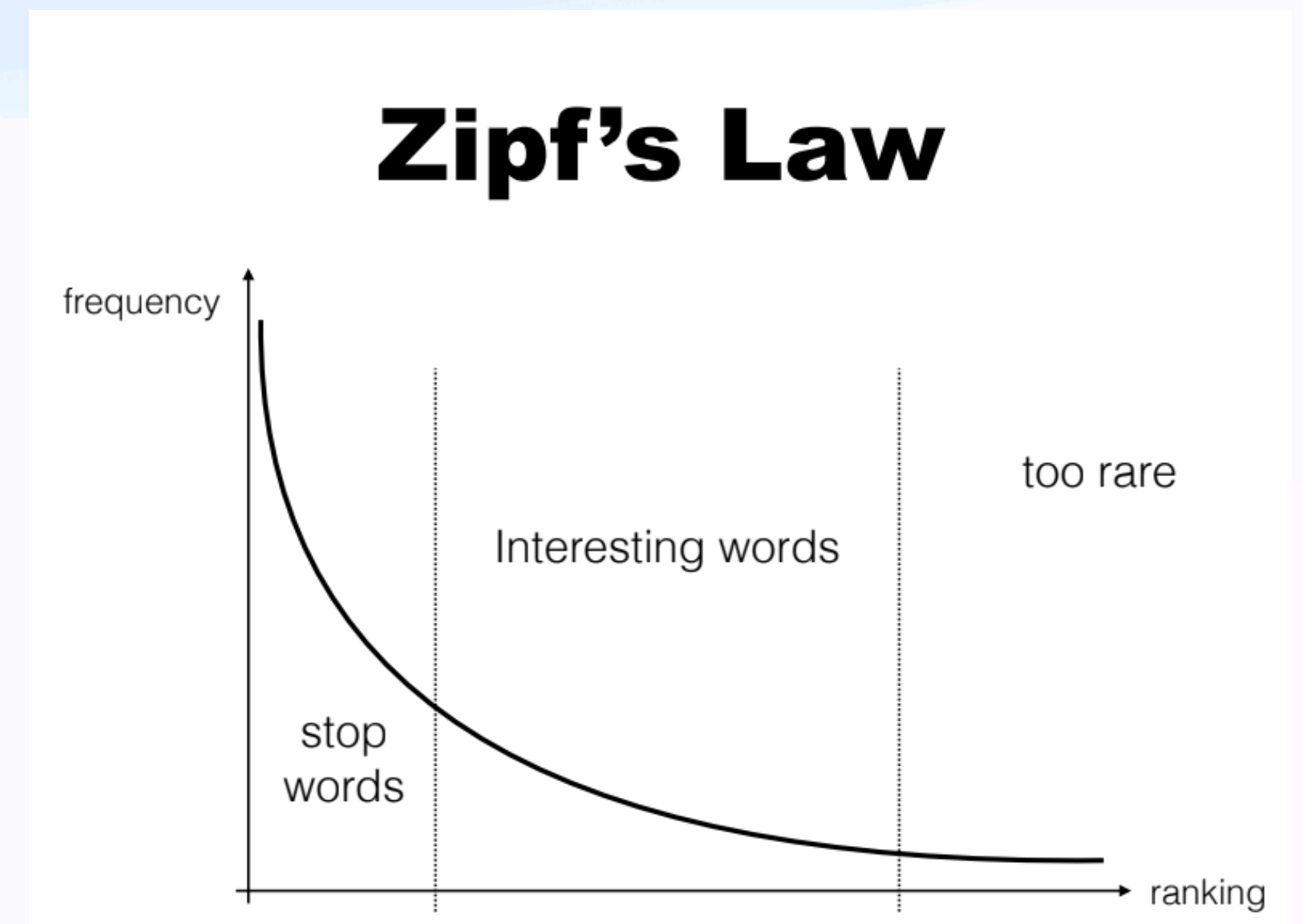
- Flux RSS / Dumps réguliers
 - Limitations dans la fréquence de mise à jour des dumps
 - Parfois des restrictions d'accès aux dumps complets
 - Besoin de gérer de gros volumes de données
- API payantes
 - Coût
 - Versions d'API qui peuvent changer

Traitement du texte

Document Understanding

Text Processing | Vocabulaire

- Quelques mots apparaissent très souvent, beaucoup de mots n'apparaissent presque jamais.
- La taille du vocabulaire augmente avec le corpus
Mais il y a de moins en moins de mots nouveaux lorsque le corpus est déjà important
- Le vocabulaire est constitué de
 - mots **bien orthographiés** (dans différentes langues)
 - mots avec **fautes d'orthographe**
 - mots **inventés** (e.g: noms de produits, de sociétés, etc.)
 - adresses mails, suites de chiffres
 - etc.



Document Understanding

Text Processing | Représentations de mots

- **Word-level**

- Avantages : Maintient l'intégrité des mots, intuitif
- Inconvénients : Large vocabulaire, mauvaise gestion des mots rares/inconnus

- **Character-level**

- Avantages : Petit vocabulaire, peut gérer n'importe quel mot
- Inconvénients : Perd le sens au niveau du mot, séquences plus longues

- **N-grams**

- Avantages : Capture le contexte local
- Inconvénients : Explosion combinatoire des possibilités

- **“Le chat mange ses croquettes”**

- Word-level

- ["le", "chat", "mange", "ses", "croquettes"]

- Character-level

- ["l", "e", " ", "c", "h", "a", "t", " ", "m", "a", "n", "g", "e", " ", "s", "e", "s", " ", "c", "r", "o", "q", "u", "e", "t", "t", "e", "s"]

- 2-grams

- ["le chat", "chat mange", "mange ses", "ses croquettes"]

Document Understanding

Text Processing | Représentations de mots

- **Tokens morphologiques**

- Avantages : Linguistiquement motivé
- Inconvénients : Nécessite des règles spécifiques à la langue

- **Tokens de sous-mots**

- Avantages : Équilibre entre taille du vocabulaire et préservation du sens
- Inconvénients : Peut créer des divisions contre-intuitives
- Les approches courantes incluent :
 - WordPiece (utilisé par BERT) : Divise les mots en unités de sous-mots communes
 - Byte-Pair Encoding (BPE, utilisé par GPT) : Fusionne itérativement les paires de caractères fréquentes

- **“Le chat mange ses croquettes”**

- Tokens morphologiques

- ["le", "chat", "mang", "#e", "ses", "croquet", "#ette", "#s"]

- Tokens de sous-mots

- Word-piece

- ["le", "chat", "mange", "ses", "cro", "##quettes"]

- BPE

- ["le", "ch", "at", "mange", "s", "es", "cro", "que", "ttes"]

Document Understanding

Text Processing | Des mots aux tokens

- Une **étape clé** dans le text processing, si la tokenization est mal réalisée, les étapes d'après vont en pâtir.
Elle est tellement importante qu'aujourd'hui on utilise souvent un **modèle entraîné** pour tokenizer.
- Considérations pratiques :
 - **La tokenisation doit être cohérente entre l'indexation et la recherche**
 - Il faut penser aux performances (temps de traitement)
 - La mémoire utilisée par l'index dépend directement des choix de tokenisation
 - Possibilité d'utiliser plusieurs types d'indexation en parallèle

Structure du document

Document Understanding

Document Structure | Supprimer le bruit contenu dans les documents

- Tâche: Boilerplate removal
- De nombreuses pages web contiennent du texte, des liens et des images qui ne sont pas directement liés au contenu principal de la page.
- Ces éléments supplémentaires sont pour la plupart du bruit et peuvent avoir un impact négatif sur le classement de la page.

WIKIPÉDIA

L'encyclopédie libre

Rechercher sur Wikipédia

Créer un compte

Sommaire [masquer]

Début

Dénominations

Évolution du langage

Description de HTML

Interopérabilité de HTML

Notes et références

Voir aussi

Hypertext Markup Language

ArticleDiscussion

LireModifierModifier le codeVoir l'historique

Le *HyperText Markup Language*, généralement abrégé **HTML** ou, dans sa dernière version, **HTML5**, est le **langage de balisage** conçu pour représenter les **pages web**.

Ce langage permet d'écrire de l'**hypertexte** (d'où son nom), de structurer **sémantiquement** une page web, de mettre en forme du contenu, de créer des formulaires de saisie ou encore d'inclure des **ressources multimédias** dont des **images**, des **vidéos**, et des **programmes informatiques**. L'HTML offre également la possibilité de créer des documents **interopérables** avec des équipements très variés et conformément aux exigences de l'**accessibilité du web**.

Il est souvent utilisé conjointement avec le **langage de programmation JavaScript** et des **feuilles de style en cascade** (CSS). HTML est inspiré du *Standard Generalized Markup Language* (SGML). Il s'agit d'un **format ouvert**.

Dénominations

[modifier | modifier le code]

L'**anglais** « *HyperText Markup Language* » se traduit littéralement en « langage de balisage d'hypertexte »¹. On utilise généralement le **sigle** « HTML », parfois même en répétant le mot « langage » comme dans « langage HTML ». *Hypertext* est parfois écrit *HyperText* pour marquer le **T** du sigle HTML.

Le public non averti parle parfois de HTM au lieu de HTML, HTM étant l'**extension de nom de fichier** tronquée à trois lettres, une limitation que l'on trouve sur d'anciens **systèmes d'exploitation** de **Microsoft**.

Évolution du langage

[modifier | modifier le code]

Durant la première moitié des années 1990, avant l'apparition des **technologies Web** comme le langage **JavaScript** (JS), les **feuilles de style en cascade** (CSS) et le *Document Object Model* (DOM), l'évolution de HTML a dicté l'évolution du *World Wide Web*. Depuis la création de l'HTML 4 en 1997, l'évolution de HTML a fortement ralenti ; dix ans plus tard, HTML 4 reste utilisé dans les **pages web**. En **2008**, la spécification du **HTML5** est à l'étude² et devient d'usage courant dans la seconde moitié des années 2010.

1989-1992 : Origine

[modifier | modifier le code]

HTML est une des trois inventions à la base du *World Wide Web*, avec le *Hypertext Transfer Protocol* (HTTP) et les **adresses web** (URL). HTML a été inventé pour permettre d'écrire des documents **hypertextuels** liant les différentes ressources d'**Internet** avec des **hyperliens**. Aujourd'hui, ces documents sont appelés « page web ». En août 1991, lorsque **Tim Berners-Lee** annonce publiquement le web sur **Usenet**, il ne cite que le langage **Standard Generalized Markup Language** (SGML), mais donne l'**URL** d'un document de suffixe **.html**.

Dans son livre *Weaving the web*³, **Tim Berners-Lee** décrit la décision de baser HTML sur SGML comme étant aussi « diplomatique » que technique : techniquement, il trouvait SGML trop complexe, mais il voulait attirer la communauté **hypertexte** qui considérait que SGML était le langage le plus prometteur pour standardiser le format des documents hypertexte. En outre, SGML était déjà utilisé par son employeur, l'**Organisation européenne pour la recherche nucléaire** (CERN). ;

Les premiers **éléments du langage HTML** comprennent :

le titre du document,

les **hyperliens**,

HTML

HyperText Markup Language

Caractéristiques

Extensions

.html, .htm

Type MIME

text/html

PUID

fmt/99

Développé par

World Wide Web Consortium & WHATWG

Version initiale

1993

Type de format

Langage de balisage

Basé sur

Standard Generalized Markup Language

Origine de

XHTML

Norme

ISO/IEC 15445

W3C HTML 4.01

W3C HTML5

ISO

15445

Spécification

Format ouvert

Site web

html.spec.whatwg.org/multipage

modifier - modifier le code - modifier Wikidata

56

Document Understanding

Les métadonnées

HTML

```
<meta name="author" content="Chris Mills" />
<meta
  name="description"
  content="The MDN Web Docs Learning Area aims to provide
complete beginners to the Web with all they need to know to get
started with developing websites and applications." />
```

HTML

```
<meta
  property="og:image"
  content="https://developer.mozilla.org/mdn-social-share.png" />
<meta
  property="og:description"
  content="The Mozilla Developer Network (MDN) provides
information about Open Web technologies including HTML, CSS, and APIs
for both websites
and HTML Apps." />
<meta property="og:title" content="Mozilla Developer Network" />
```

Document Understanding

Les métadonnées - Open Graph

```
HTML
<meta
  property="og:image"
  content="https://developer.mozilla.org/mdn-social-share.png" />
<meta
  property="og:description"
  content="The Mozilla Developer Network (MDN) provides
information about Open Web technologies including HTML, CSS, and APIs
for both websites
and HTML Apps." />
<meta property="og:title" content="Mozilla Developer Network" />
```

- Balises Open Graph obligatoires (si on utilise les metas og)
- og:title —> le titre de notre noeud
- og:type —> exemple: “video.movie”, en fonction du type, certaines autres propriétés doivent être spécifiées
- og:image —> l’url de l’image qui représente notre noeud
- og:url —> l’url de notre noeud, utilisée comme ID
- Pour aller plus loin

Document Understanding

Les métadonnées - JSON LD

- JSON - LD pour Linked Data
- Pour lier les documents web entre eux

```
{
  "@context": "https://json-ld.org/contexts/person.jsonld",
  "@id": "http://dbpedia.org/resource/John_Lennon",
  "name": "John Lennon",
  "born": "1940-10-09",
  "spouse": "http://dbpedia.org/resource/Cynthia_Lennon"
}
```

- Exemple avec Ensai.fr

Analyse des liens

Document Understanding

Analyse des liens

- Le crawler s'aide des liens entre les pages pour trouver de nouvelles pages à crawler
- Ces liens nous permettent
 - de créer un graphe du web
 - de comprendre les relations entre les pages
 - d'ordonner les pages en fonction de leur importance

Document Understanding

Analyse des liens

HTML



```
<p>
  I'm creating a link to
  <a href="https://www.mozilla.org/en-US/">the Mozilla homepage</a>.
</p>
```

HTML

</> Play



```
<a href="https://developer.mozilla.org/en-US/">
  
</a>
```

Document Understanding

Analyse des liens

```
HTML </> Play  
  
<p>  
  I'm creating a link to  
  <a  
    href="https://www.mozilla.org/en-US/"  
    title="The best place to find more information about Mozilla's  
      mission and how to contribute">  
    the Mozilla homepage</a>.  
</p>
```

- Le titre est seulement vu par l'utilisateur quand la souris passe dessus

```
HTML </>  
  
<p>  
  Want to write us a letter? Use our  
  <a href="contacts.html#Mailing_address">mailing address</a>.  
</p>
```

- Vers un fragment de la page

Document Processing

Analyse des liens | Le texte des ancres

- En général, un texte court qui décrit la page vers laquelle on va cliquer
Il peut avoir les memes caractéristiques qu'une requête web
- Extraire le texte des ancres nous permet ensuite de matcher ce texte avec la requête
- Peut être très utile pour les abréviations:
YouTube.com -> yt, ytb
- Mais doit etre nettoyé:
"Cliquez ici", "check here", "découvrir"

Document Processing

Analyse des liens | Le texte des ancres

HTML



```
<p><a href="https://www.mozilla.org/firefox/">Download Firefox</a></p>
```



HTML



```
<p>  
  <a href="https://www.mozilla.org/firefox/">Click here</a> to download  
  Firefox  
</p>
```



Document Processing

Analyse des liens | La popularité d'une page

- La majeure partie du web est composée de pages assez peu intéressantes, datées, ou de pages que l'on qualifie de spam
- On va donner un ou plusieurs score de qualité, de confiance aux urls
- Le plus connu: **Page Rank**

Document Processing

Analyse des liens | Page rank

- **Calcul du score de popularité d'une URL**
- **Principe** : Plus une URL A reçoit d'inlinks, plus elle est populaire.
- **Poids des liens** : Les liens provenant de pages populaires comptent plus. Ainsi, une page A ne sera pas favorisée si elle reçoit des liens de pages peu populaires ou de spam.
- **Objectif** : Calculer un score de popularité en prenant en compte la qualité des pages pointant vers A.
- Mots-clés :
 - Page = Document = Site = URL
 - Inlinks : Liens pointant vers une page
 - Outlinks : Liens d'une page vers d'autres pages

Document Processing

Analyse des liens | Page rank

- Algorithme “Random surfer model”
- **Principe** : Le modèle simule un utilisateur qui navigue au hasard sur le web.
- **Threshold** (ex. 0.15) : Contrôle la probabilité de choisir une page aléatoire, laissant une majorité des choix aux liens internes.
- **PageRank** : Le score de PageRank d'une page est la probabilité qu'un surfer arrive sur cette page.

```
func random_surfer_model(threshold)
  r <- random(0,1)
  if r < threshold
    choisis une page web aléatoire
  else
    clique sur un lien dans la page actuelle
  random_surfer_model(threshold)
```


Document Processing

Analyse des liens | Page rank

- L'algorithme tourne indéfiniment et explore plusieurs fois les sites, mais avec une probabilité plus élevée pour les sites populaires.

⚛ Problème sans le choix aléatoire ?

Document Processing

Analyse des liens | Page rank

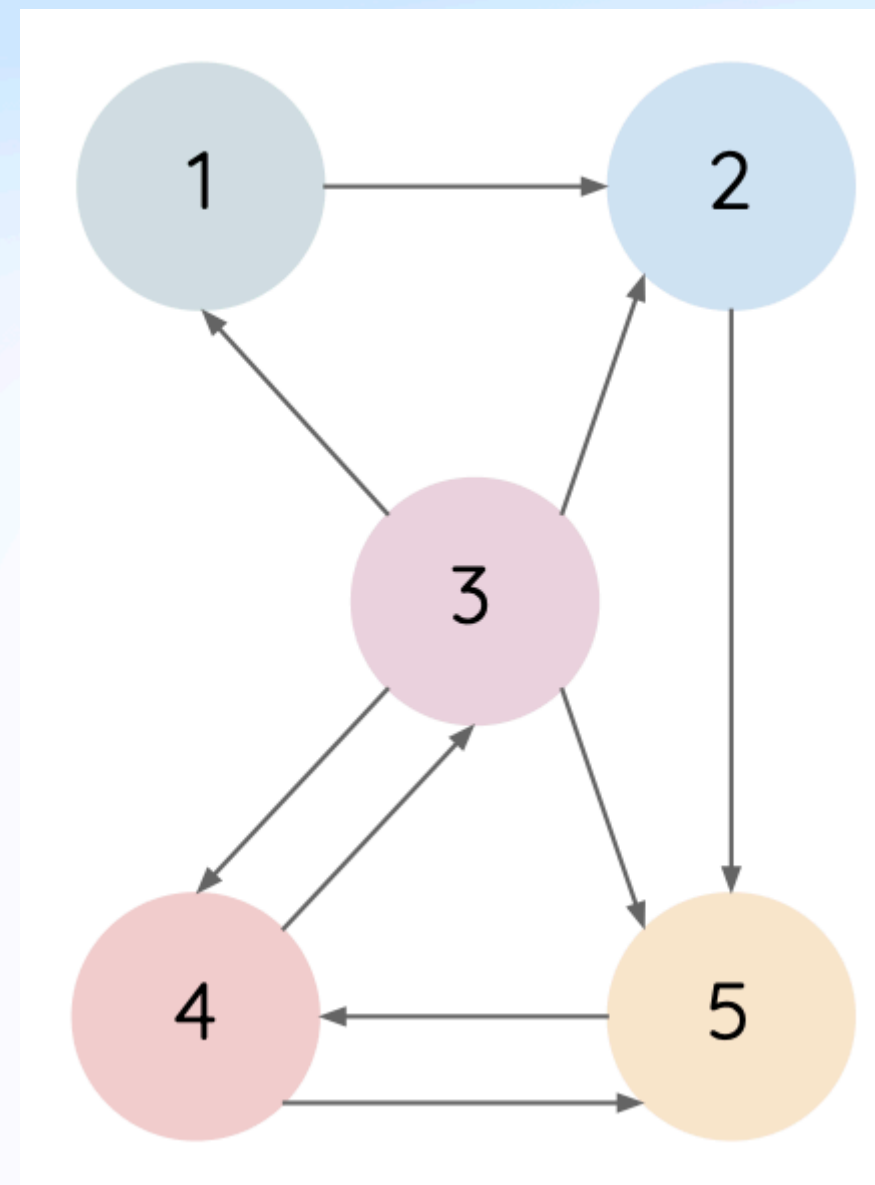
- L'algorithme tourne indéfiniment et explore plusieurs fois les sites, mais avec une probabilité plus élevée pour les sites populaires.
- Problème sans le choix aléatoire ?
 - L'algorithme pourrait se retrouver bloqué sur :
 - Des pages sans liens
 - Des pages dont les liens sont cassés
 - Des pages dans des boucles infinies

Document Processing

Analyse des liens | Page rank

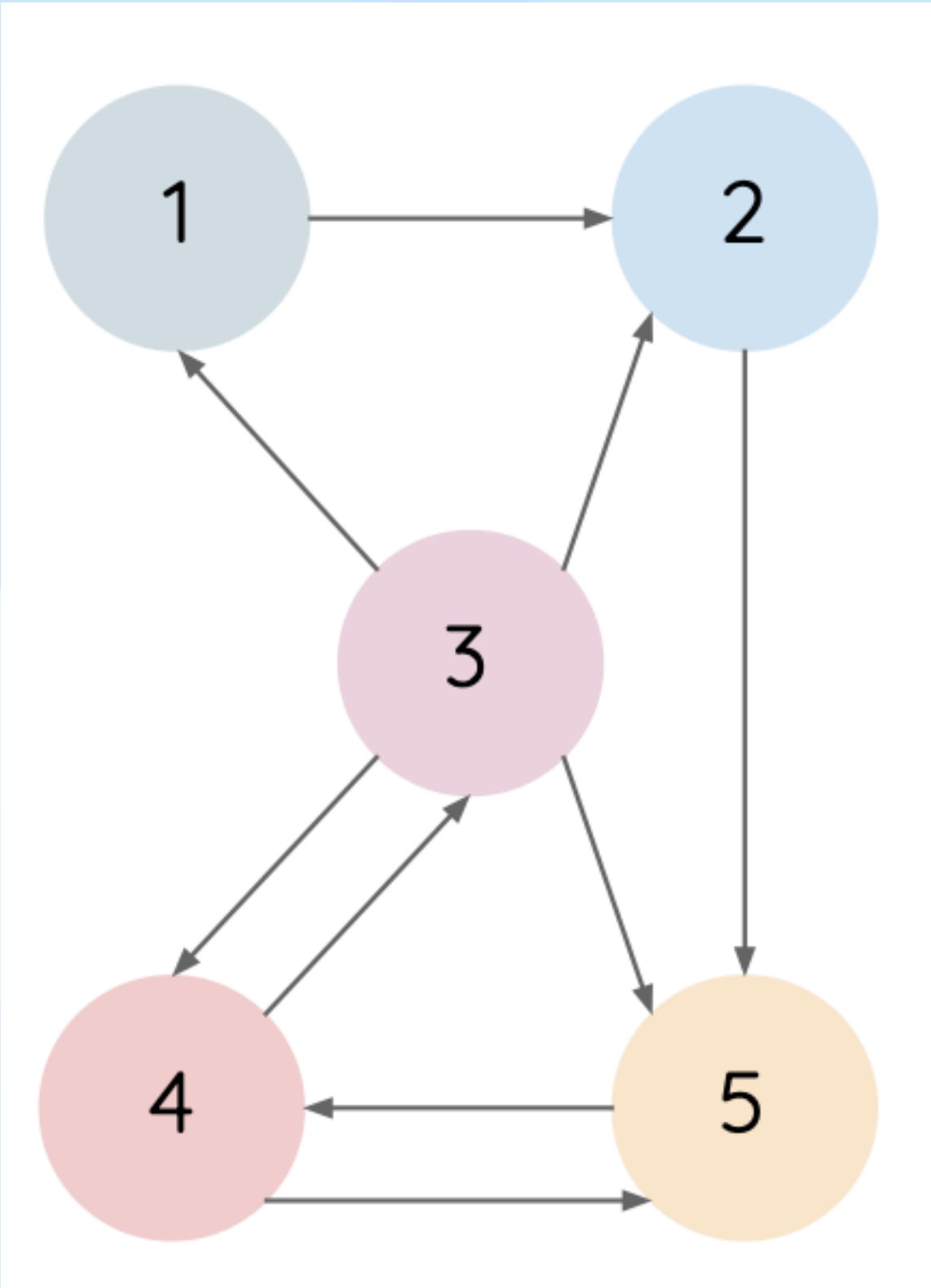
- Au départ toutes les pages ont le même page rank

✳️ Quelle serait la page la plus populaire dans cet exemple?



Document Processing

Analyse des liens | Page rank



	Iteration 0	Iteration 1	Iteration 2	Final Rank
P_1	$1/5$	$1/20$	$1/40$	5
P_2	$1/5$	$5/20$	$3/40$	4
P_3	$1/5$	$1/10$	$5/40$	3
P_4	$1/5$	$5/20$	$15/40$	2
P_5	$1/5$	$7/20$	$16/40$	1

Document Processing

Analyse des liens | Page rank

$$PR(u) = \frac{\lambda}{N} + (1 - \lambda) \cdot \sum_{v \in B_u} \frac{PR(v)}{L_v}$$

- N -> le nombre de documents
- λ -> probabilité de choisir une page random
 $1 - \lambda$ -> probabilité de cliquer sur un lien
- B_u -> inlinks de l'url u
- L_v -> # outlinks depuis l'url v

Document Embeddings

Document Embeddings

Dense representations | BERT-like embeddings

- Embeddings: représentation vectorielle d'une string (plus ou moins longue)
- BERT-like embeddings: représentation contextualisée au niveau du **word-piece**
Exemple:
 - Danone est mon yaourt préféré
 - Je déteste DanoneLe vecteur de Danone sera différente
- **Problèmes en recherche d'information:**
 - Ce sont des representation assez **longues à générer**
Comme elles dependent du contexte, on ne peut pas simplement avoir une hashmap de token -> vecteur
 - Ces embeddings ont une contrainte de taille de la string en input, **un document web est souvent bien plus long**
On choisit alors
 - soit de représenter le début du document
 - soit de diviser le document en chunks de max 512 wp et de multiplier les vecteurs résultants
 - Soit de partir d'un summary du document

Document Embeddings

Dense representations | ColBERT

- Similarité entre la requête et le document
- Un même modèle qui encode la requête et le document séparément et donne un score de similarité entre un document et une requête
- On va encoder le document au moment de l'indexation
On n'aura plus qu'à encoder la requête

Partie 2

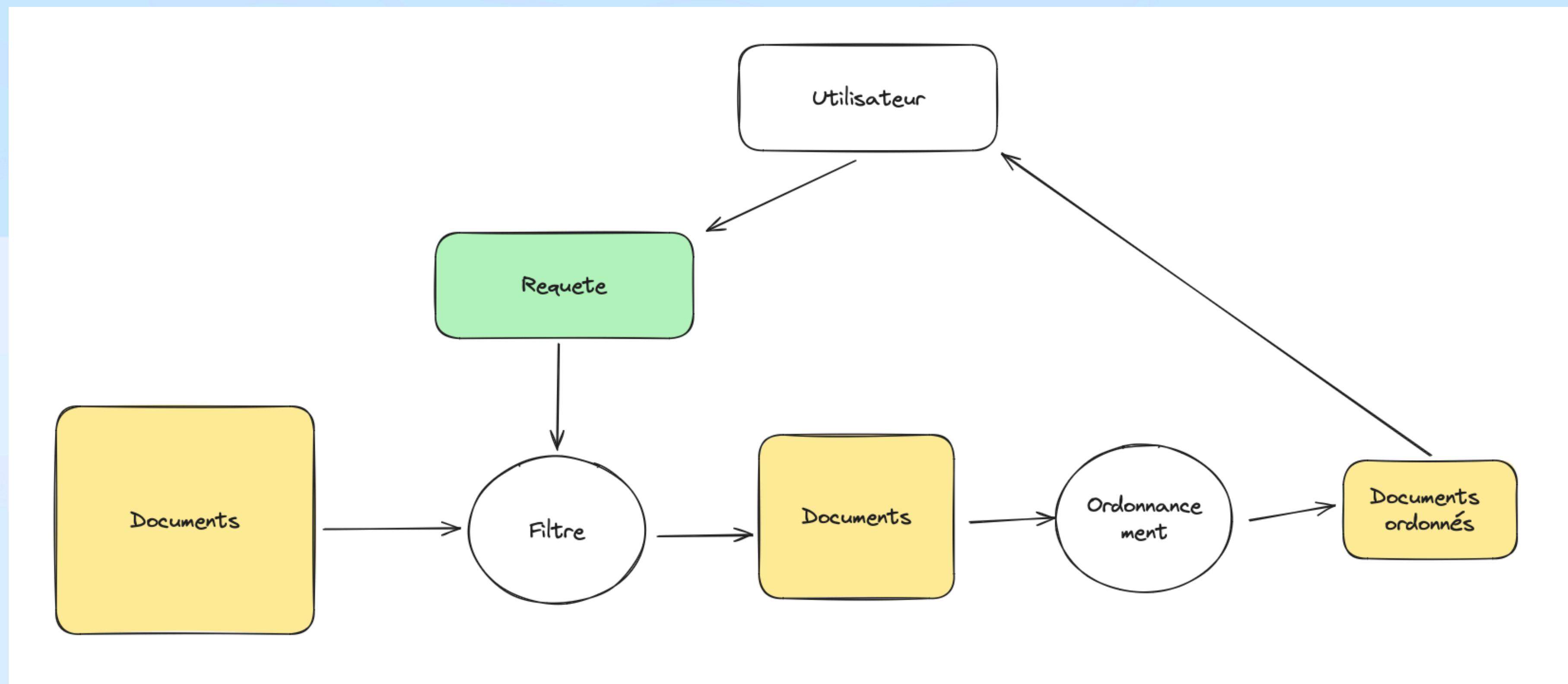
Traitement de la requête et ordonnancement des résultats

Rappels

Traitement de la requête et ordonnancement des résultats

- A ce stade, on a:
 - Une base de données avec des milliards de documents
 - Ces documents ont été process pour en extraire plusieurs informations:
 - Textuelles: title, content, metatags, headers etc.
 - Scores: popularité (ex: page rank), confiance, spam etc.
 - Induites: langue, topic etc.

Traitement de la requête et ordonnancement des résultats



Partie 2

Traitement de la requête et ordonnancement des résultats

Index

Index

- On va parler d'index, de la base de donnée qui contient les documents
- Il faut garder en tête que l'on veut qu'ils matchent avec la requête
Toute la construction d'un index textuel est faite par rapport au processing de la requête
- La structure de données la plus commune est appelée **index inversé** (ou inverted index)

Index inversé

- Un index inversé est une structure de données fondamentale en recherche d'information, comparable à l'index que l'on trouve à la fin d'un livre
- Dans un **livre traditionnel**, on peut trouver un **index** :
 - L'index liste les mots importants par ordre alphabétique
 - Chaque mot pointe vers les numéros de pages où il apparaît
 - Seuls les termes jugés significatifs sont indexés
- Dans un **index inversé** (moteur de recherche) :
 - Il répertorie TOUS les mots (tokens) du corpus
 - Chaque token est associé à la liste des documents qui le contiennent
 - La structure est token → [document1, document2, ...]
 - L'exhaustivité est privilégiée sur la sélectivité

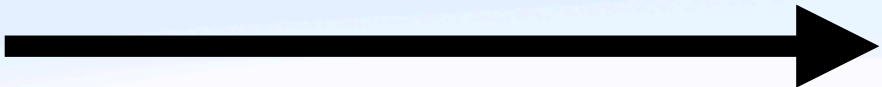
Création d'un index inversé

Document 1

Paul mange une pomme
dans le jardin

Document 2

Jean mange une poire
dans la cuisine



Index	inversé
Tokens	Documents
Paul	1
mange	1
une	1
pomme	1
dans	1
le	1
jardin	1
Jean	2
mange	2
une	2
poire	2
dans	2
la	2
cuisine	2

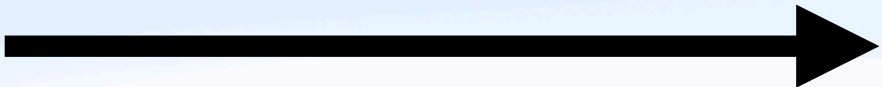
Création d'un index inversé

Document 1

Paul mange une pomme
dans le jardin

Document 2

Jean mange une poire
dans la cuisine



Index	inversé
Tokens	Documents
Paul	1
mange	1,2
une	1,2
pomme	1
dans	1,2
le	1
jardin	1
Jean	2
poire	2
la	2
cuisine	2

Création d'un index inversé

Avec un compte de chaque token

Document 1

Paul mange la pomme
dans la chambre

Document 2

Jean mange la poire
dans la cuisine

Index	inversé
Tokens	Documents
Paul	1:1
mange	1:1,2:1
la	1:2,2:2
pomme	1:1
dans	1:1,2:1
chambre	1:1
Jean	2:1
poire	2:1
cuisine	2:1

Création d'un index inversé

Avec la position de chaque token

Document 1

Paul mange la pomme
dans la chambre

Document 2

Jean mange la poire
dans la cuisine

Index	inversé
Tokens	Documents
Paul	1:[0]
mange	1:[1],2:[1]
la	1:[2,5],2:[2,5]
pomme	1:[3]
dans	1:[4],2:[4]
chambre	1:[6]
Jean	2:[0]
poire	2:[3]
cuisine	2:[6]

Création d'un index inversé

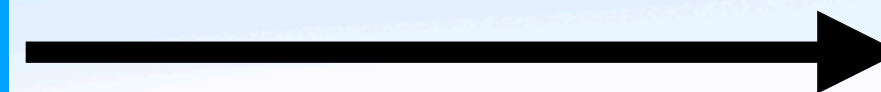
N-grams

Document 1

Paul mange la pomme
dans la chambre

Document 2

Jean mange la poire
dans la cuisine



Index

inversé

Tokens

Documents

Paul mange	[1]
mange la	[1, 2]
la pomme	[1]
pomme dans	[1]
dans la	[1, 2]
la chambre	[1]
Jean mange	[2]
la poire	[2]
poire dans	[2]
la cuisine	[2]

Index inversé

Compression

- Challenge
 - Les listes inversées (par token/n-gram) croissent exponentiellement
 - Impact sur le stockage disque et la mémoire vive
- Solutions de Compression
 - Encodage des différences (Delta encoding)
 - Stocker les écarts entre IDs plutôt que les IDs complets
 - Ex: [100, 102, 105] → [100, 2, 3]
 - Compression par blocs
 - Regrouper les IDs en blocs compressés
 - Décompression rapide à la demande

Plusieurs index, plusieurs champs

- Pourquoi des Index Multiples ?
 - Les documents sont structurés (titres, paragraphes, métadonnées)
 - Chaque champ a une importance différente
 - Un index par champ permet une recherche granulaire
- Avantages
 - Recherche ciblée par champ
 - Pondération des champs dans le scoring
 - Préservation de la structure logique

Plusieurs index, plusieurs champs

- Exemple:
 - Requete: le bon coin
 - Document 1 -
 - title : Le bon coin
 - content : Trouvez tout ce dont vous avez besoin
 - Document 2 -
 - title : La déco de Lucette
 - content: Je trouve tous mes meubles sur le bon coin!

```
document {  
  titre: index_titre,  
  contenu: index_contenu,  
  auteur: index_auteur,  
  date: index_date  
}
```

Plusieurs index, plusieurs champs

Option 1: plusieurs index inversés

- L'idée est de construire plusieurs index inversés,
- Un par champs textuel :
 - Title
 - Content
 - Les champs de l'url (domain, path)
- Si on cherche les tokens de la requête dans le titre, on peut directement taper dans l'index titre. Mais en général, on veut chercher dans plusieurs champs et avec cette méthode, on doit récupérer les listes inversées depuis plusieurs index.

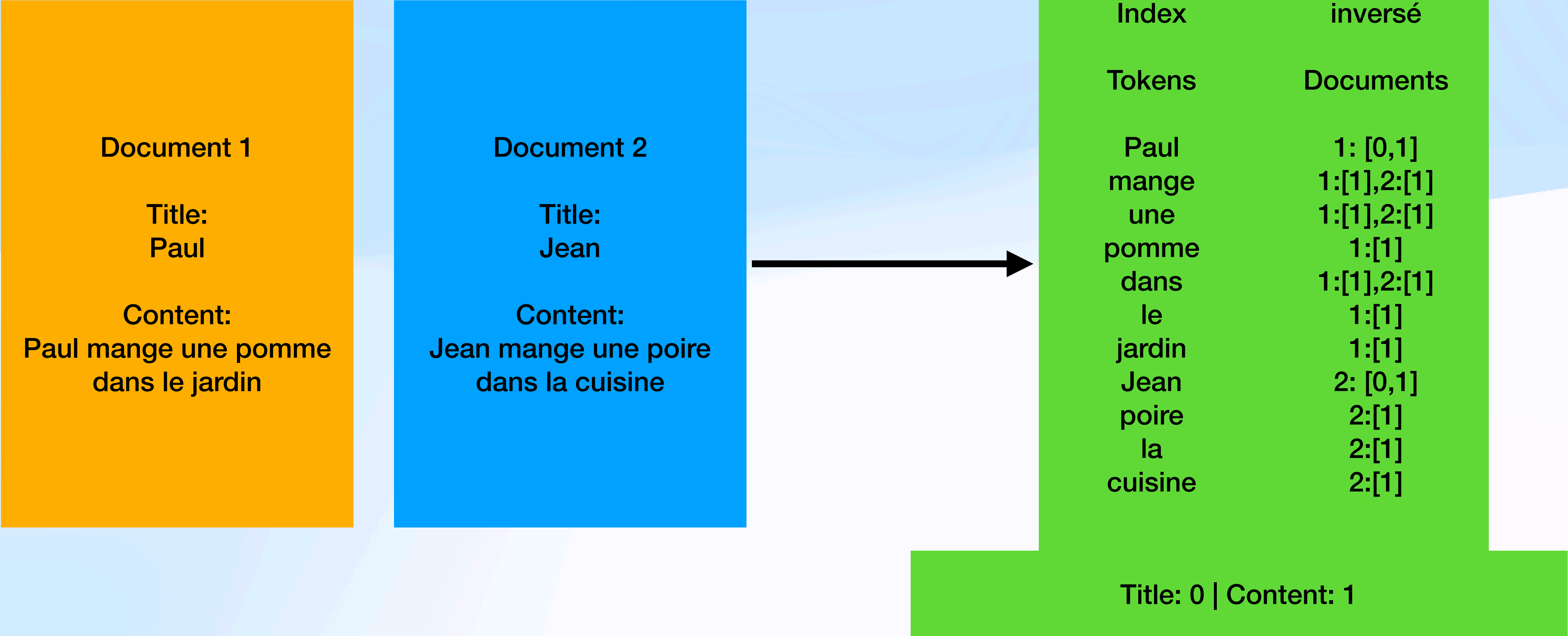
Plusieurs index, plusieurs champs

Option 1: plusieurs index inversés

- Avantages :
 - Recherche et filtres rapides par champ spécifique
 - Mise à jour simplifiée d'un seul champ
 - Optimisation des index par type de contenu
 - Flexibilité des schémas par champ
- Inconvénients :
 - Consommation mémoire/disque plus importante
 - Maintenance de multiples structures
 - Complexité accrue pour les requêtes multi-champs

Document fields

Option 2: one index to rule them all



Plusieurs index, plusieurs champs

Option 2: one index to rule them all

- Avantages :
 - Structure plus simple
 - Recherche unifiée
 - Économie d'espace
- Inconvénients :
 - Listes plus longues
 - Complexité accrue du scoring

Document fields

Option 3: ajout de l'information de position

Maison	Inverted index with word positions: [document, word position] [1,2] [1,4] [2,7] [2,18] [2,23] [3,2] [3,6] [4,3] [4,13]
Title	Positions of title in the document [document, (begin title, end title)] [1: (1,3)] [2: (1,5)] [4: (9,15)]

- On peut étendre ce concept à tous les champs et même définir de nouvelles listes comme la position des verbes dans le document, ou des entités nommées

Construction d'un index inversé

Algorithme

- Entrées
 - Documents à indexer
 - Options de traitement (lowercase, stemming, etc.)
- Sortie
 - Index inversé {token -> [doc_ids]}
- Pour chaque document / champs :
 - Traiter le texte (tokenization, stemming etc.)
 - Pour chaque token
 - Ajout dans l'index

Constructions de l'index

Limites de cet algorithme

- Fonctionne pour des applications très simples, sur peu de documents (quelques milliers)
- Limite 1: il garde en mémoire toutes les listes inversées
- Limite 2: compliqué à paralléliser en tant que tel avec un index que l'on accède en permanence

Constructions de l'index

Solutions: merging + distributed programming

- Pour régler le problème de mémoire: fusion des index

Index A	Cuisine	2 3 4 5		Pomme	2 4		
Index B	Cuisine		6 9			Poire	15 20 22
Index final	Cuisine	2 3 4 5 6 9		Pomme	2 4	Poire	15 20 22

Constructions de l'index

Solutions: merging + distributed programming

- Pour régler le problème de parallélisation: Map Reduce
 1. MAP: transforme l'input en paires clé:valeur
 2. SHUFFLE:
Partitionne les données par token
Hash(token) → machine_id
Regroupe (token, [doc_info1, doc_info2, ...])
 3. REDUCE: traite les données par batch, où toutes les paires ayant la même clé sont traitées en même temps.

```
ENTRÉE: document
SORTIE: (token, (doc_id, position))

FONCTION map(document):
    tokens ← tokenize(document)
    POUR CHAQUE token, position DANS tokens
        ÉMETTRE (token, (document.id, position))
```

```
ENTRÉE: (token, [(doc1, pos1), (doc2, pos2), ...])
SORTIE: (token, posting_list)

FONCTION reduce(token, doc_positions[]):
    posting_list ← []
    POUR CHAQUE (doc, pos) DANS doc_positions
        AJOUTER (doc, pos) À posting_list
    RETOURNER (token, posting_list)
```


Put / update

- Jusqu'à présent, nous avons supposé que la collection de documents était statique.
- Dès qu'on ajoute un nouveau document:
 - De nouveaux tokens doivent être ajoutés
 - Les listes pour les tokens déjà présents doivent être mises à jour

Put / update

- Une des solutions: maintenir deux index :
 - un grand index principal
 - un petit index auxiliaire qui stocke les nouveaux documents, il est conservé en mémoire.
- Les recherches sont effectuées sur les deux index et les résultats sont fusionnés.
- Les suppressions sont stockées dans une liste qui nous servira de filtre
- Les documents sont mis à jour en les supprimant et en les réinsérant.
- Dès que l'index auxiliaire est trop grand, on le fusionne avec le grand et on en crée un nouveau pour les prochaines opérations.

Partie 2

Traitement de la requête et ordonnancement des résultats

Compréhension de la requête

Besoin de l'utilisateur

- 1. Informationnelle

- L'utilisateur cherche à apprendre ou comprendre
 - "tarte aux pommes"
 - "Symptômes de la grippe"
 - "Qui a inventé Internet"

- 2. Transactionnelle

- L'utilisateur veut réaliser une action ou transaction
 - "Acheter iPhone 15 Pro"
 - "Réserver hôtel Paris"
 - "Télécharger Adobe Photoshop"

- 3. Navigationnelle

- L'utilisateur cherche un site ou une page spécifique
 - "Facebook login"
 - "Gmail connexion"
 - "Twitter Elon Musk"
- La compréhension de ces intentions permet d'adapter :
 - Le **nombre** de résultats à afficher
 - Le **type** de contenu à prioriser
 - **L'interface de présentation des résultats**
-

Besoin de l'utilisateur

Défis de l'Interprétation des Requêtes

Diversité des Besoins

- Une même requête peut avoir plusieurs intentions
 - "Python" → langage de programmation ou serpent ?
- Nécessite des stratégies d'algorithmes adaptées
 - Désambiguïsation contextuelle
- Diversification des résultats

Écart Besoin-Expression

1. Difficultés d'Expression

- Vocabulaire limité
- Manque d'expertise du domaine
- Barrière linguistique

2. Contraintes Interface

- Format court privilégié
- Pas de structure imposée
- Absence de contexte explicite

Exemple

Requête : "java"

- Développeur : recherche documentation
- Étudiant : cours débutant
- Voyageur : île de Java
- Amateur café : variété de café

Types de requêtes

Compréhension des Besoins d'Information

1. Complexité de l'Interprétation

- Les requêtes sont souvent ambiguës
 - Même mot, sens différents
 - Contexte manquant
 - Intentions multiples
- L'historique de navigation peut aider
 - Requêtes précédentes
 - Pages visitées
 - Profil utilisateur

2. Challenges de l'Expression

Côté Utilisateur

- Difficulté à formuler précisément le besoin
- Manque de connaissance du vocabulaire technique
- Incertitude sur ce qui est recherché exactement

Contraintes Techniques

- Interfaces privilégiant les requêtes courtes
- Absence de structure formelle
- Limitation du nombre de mots

Le moteur doit donc proposer un mix de résultats couvrant ces différents aspects.

Types de requêtes

Compréhension des Besoins d'Information

3. Impact sur le Moteur de Recherche

Techniques d'Amélioration

- Suggestion de requêtes
- Correction orthographique
- Extension de requête
- Reconnaissance d'entités

Stratégies de Ranking

- Diversification des résultats
- Personnalisation du classement

- Prise en compte du contexte

4. Exemple

Requête simple : "voiture électrique"

- plusieurs compréhensions possibles:
 - Achat : prix, modèles
 - Information : fonctionnement, avantages
 - Comparaison : différentes marques
 - Actualités : dernières innovations

Opérateurs dans une requête

Les opérateurs de recherche Google

- Les opérateurs de recherche permettent d'affiner vos requêtes Google pour obtenir des résultats plus précis. Voici les principaux opérateurs à connaître :
 - **Recherche exacte**
 - Utilisez les guillemets `"..."` pour rechercher une expression exacte
 - - Exemple : `"marketing digital"` trouvera uniquement les pages contenant exactement cette expression
 - **Opérateurs logiques**
 - **OR (OU)**
 - Pour trouver des pages contenant l'un ou l'autre
 - Exemple : ``marketing OR communication`` trouvera des pages contenant soit "marketing", soit "communication"
 - **Exclusion (-)**
 - Pour exclure les pages contenant ce terme
 - Exemple : ``marketing -digital`` trouvera des pages sur le marketing mais exclura celles parlant de marketing digital
 - **Opérateurs de localisation**
 - **inurl:**
 - Recherche un terme spécifiquement dans l'URL de la page
 - Exemple : ``inurl:blog`` trouvera des pages dont l'URL contient le mot "blog"

Google's query operators

Transformations de la requête

Transformations de la requête

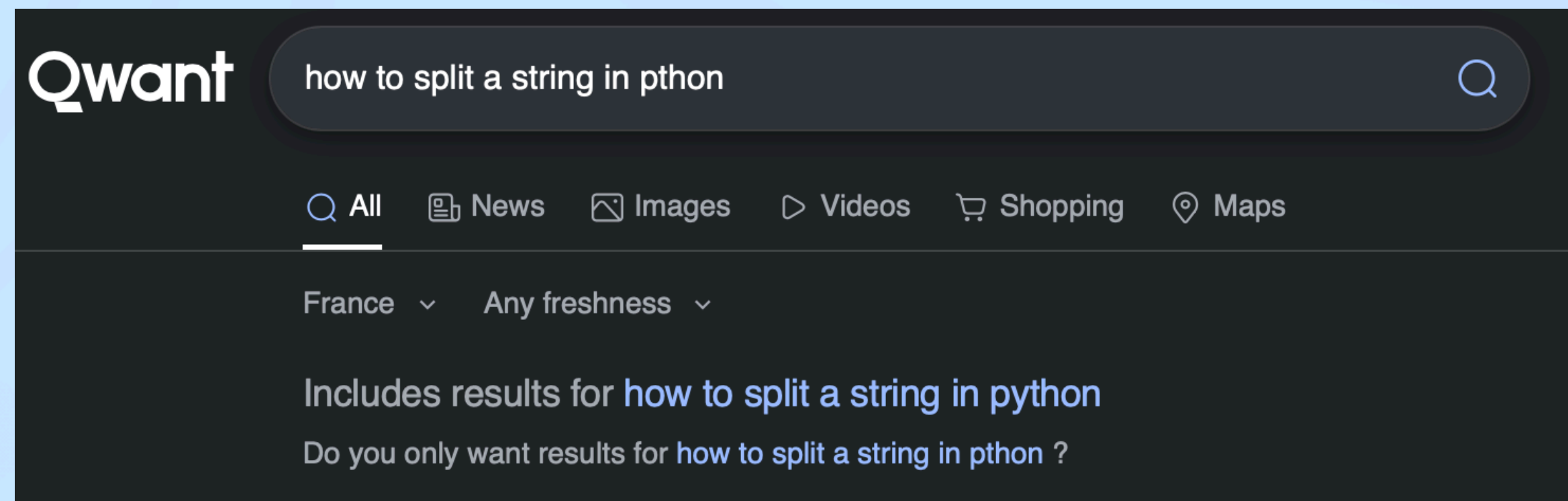
Méthodes de reformulation de requêtes

- Méthodes globales

- Les méthodes globales modifient ou étendent les termes d'une requête de manière **indépendante des résultats**. Cette reformulation est basée uniquement sur les propriétés linguistiques et sémantiques des termes.
- 1. **Stemming** (Racinisation)
 - Réduit les mots à leur racine ou forme canonique
 - Exemple :
 - "chanter", "chanteur", "chantant" → "chant" -> en français
 - "marketing", "marketeur", "marketed" → "market" -> en anglais
 - Permet de trouver des documents contenant différentes formes d'un même mot
- 2. **Correction orthographique**
 - Détecte et corrige automatiquement les erreurs de frappe et d'orthographe
 - Exemple :
 - "marketting" → "marketing"
 - "developement" → "development"
 - Inclut les variations courantes et les fautes fréquentes

Transformations de la requête

Méthode globale | Types d'erreurs orthographiques courantes



- 1. Erreur d'insertion

- Définition : Ajout d'une lettre supplémentaire dans le mot
- Exemple : "vaccances" → "vacances"

- 2. Erreur de suppression

- Définition : Omission d'une lettre dans le mot
- Exemple : "redit" → "reddit"

- 3. Erreur de substitution

- Définition : Remplacement d'une lettre par une autre
- Exemple : "youyube" → "youtube"

- 4. Erreur de transposition

- Définition : Inversion de l'ordre de deux lettres adjacentes
- Exemple : "frnace info" → "france info"

- Applications pratiques

- Ces catégories d'erreurs sont utilisées dans les algorithmes de correction automatique
- Elles permettent de suggérer des corrections en fonction du type d'erreur identifié
- Par exemple, la distance de Levenshtein peut calculer le nombre minimal d'opérations (insertion, suppression, substitution) nécessaires pour passer d'un mot à un autre

Transformations de la requête

Méthode globale | Correction orthographique

- Méthode de base
- **1. Construction du vocabulaire de référence**
 - Source : extraction à partir de l'index existant
 - Critères de sélection :
 - Inclusion des mots fréquents
 - Exclusion des termes rares (potentiellement mal orthographiés eux-mêmes)
 - Focus sur les termes de confiance élevée
- **2. Processus de correction**
 - Détection : identification des mots absents du vocabulaire
- Correction : recherche du terme le plus proche dans le vocabulaire
- Mesure de similarité :
 - Distance de Levenshtein
 - Distance de Hamming
 - Similarité phonétique
- **3. Limitations**
 - Coût computationnel élevé
 - Risque de fausses corrections
 - Complexité des règles linguistiques
 - **Dépendance à la qualité du vocabulaire**

Transformations de la requête

Méthode globale | Correction orthographique

- Solution : l'autocomplétion des requêtes

- **Avantages**

- Prévention des erreurs en amont
- Réduction de la charge de correction
- Meilleure expérience utilisateur

- **Mise en œuvre**

- Indexation préalable des termes fréquents
- Suggestion dès les premières frappes
- Classement par popularité/pertinence
- Mise à jour dynamique des suggestions

- **Architecture recommandée**

- 1. Autocomplétion comme première ligne de défense
- 2. Correction orthographique en solution de secours
- 3. Apprentissage continu basé sur les interactions utilisateur

- **Bonnes pratiques**

- Maintenir le vocabulaire à jour
- Garder les logs des corrections pour amélioration continue
- Permettre à l'utilisateur de valider les corrections proposées

Transformations de la requête

Méthode globale | Segmentation de la requête

France 2 direct

tour de france 2023

Kayak

Argentine france

programme tv ce soir

Coupe du monde de rugby

météo paris

Avatar 2 film

Nice Marseille train

bon coin

Atelier solidaire à
Dignes les bains

Jean luc godard

Reine d'angleterre

Ouest France

Voiture occasion

cnews direct

Transformations de la requête

Méthode globale | Segmentation de la requête

Input: new york times square dance

Explication 1: The New York Times is a journal, and a square dance is a dance for four couples, arranged in a square.

Output 1: [new york times] [square dance]

Explication 2: New York is a city, Times Square a famous place in this city, and it's possible people would dance there.

Output 2: [new york] [times square] dance

Transformations de la requête

Méthode globale | NER sur la requête

France 2 direct

tour de france | 2023

Kayak

Argentine | france

programme tv | ce soir

Coupe du monde de rugby

météo | paris

Avatar 2 | film

Nice | Marseille | train

bon coin

Atelier solidaire à |
Dignes les bains

Jean luc godard

Reine d'angleterre

Ouest France

Voiture occasion

cnews direct

Transformations de la requête

Méthode globale | NER sur la requête

Types d'entités:

- Personne
 - Entreprise
 - Objet
 - Marque
 - Lieu
 - Événement
 - Service
 - Action (verbe)
 - Tech (langage, framework, etc.)
- etc.

Transformations de la requête

Méthodes locales | Query expansion

- On cherche à matcher le plus de documents pertinents possibles
- La tâche de query expansion se matérialise par différentes techniques:
 - Shingle / ngram:
Requete: **le bon coin**, on va alors aussi vouloir chercher dans l'url **leboncoin / lebon boncoin**
 - Termes liés:
Requête: **yt**, on va aussi vouloir chercher YouTube
 - Stemming peut être vu comme une technique d'expansion de la requête

Transformations de la requête

Méthodes locales | Query expansion

Approches généralement basées sur une analyse de la **co-occurrence des termes**

- soit dans la collection entière de documents,
- soit dans une grande collection de requêtes,
- soit dans les documents les mieux classés dans une liste de résultats.

On va chercher à trouver les tokens associés dans nos documents aux mots dans la requete:

Exemple: aquarium pourrait etre associé à zoologie, poisson, reptile, corail, animal, mollusque, marin, sous-marin, parc etc.

Transformations de la requête

Méthodes locales | Query expansion

Toutes les techniques qui reposent sur l'analyse des documents sont confrontées à des problèmes de calcul et de précision en raison de la taille de l'index et de la variabilité de la qualité des documents.

Au lieu de s'aider des informations contenues dans l'index, on peut partir de **logs de requêtes**

La tâche de query expansion consiste alors à **trouver des requêtes similaires**

Transformations de la requête

Méthodes locales | Relevance feedback

- Impliquer l'utilisateur afin d'améliorer l'ensemble des résultats finaux.
- L'utilisateur donne son avis sur la pertinence ou non des documents dans un ensemble initial de résultats.
 - L'utilisateur lance une requête dans le moteur
 - Le moteur renvoie un ensemble initial de résultats de recherche lié à cette requête
 - L'utilisateur annote certains documents retournés comme pertinents ou non pertinents.
 - Le système calcule une meilleure représentation du besoin d'information en fonction des commentaires de l'utilisateur
 - Le système affiche un ensemble révisé de résultats de recherche

Transformations de la requête

Méthodes locales | Relevance feedback

- **On peut lancer plusieurs itérations de ce process**, jusqu'à ce que l'utilisateur soit pleinement satisfait des résultats présentés
- On peut aussi utiliser ces informations pour suivre l'évolution du besoin d'information d'un utilisateur : le fait de voir certains documents peut amener les utilisateurs à affiner sa compréhension des informations qu'il recherche
- **Dans le web**, on utilise peu cette technique:
difficile à expliquer à l'utilisateur moyen, qu'il comprenne qu'il aura une meilleure réponse si il nous informe de ce qui est pertinent ou non dans les résultats qu'il voit

Transformations de la requête

Méthodes locales | Pseudo-relevance feedback

- Pseudo-relevance feedback = Relevance feedback automatisé
- En général on utilise les clicks des utilisateurs en inférant que si l'utilisateur a cliqué sur un document, il est pertinent.
- Mais comme toutes les méthodes d'annotation automatiques, elles apportent des biais :

Transformations de la requête

Méthodes locales | Pseudo-relevance feedback

- Relevance feedback automatisé
- En général on utilise les clicks des utilisateurs en inférant que si l'utilisateur a cliqué sur un document, il est pertinent.
- Mais comme toutes les méthodes d'annotation automatiques, elles apportent des biais :
 - Qu'est-ce qu'il se passe si tous les documents sont non pertinents ?
 - Qu'est-ce qu'il se passe si la requête est ambiguë et que le moteur se focus sur un seul des sens ?

Transformations de la requête

Méthodes de reformulation de requêtes

- 3. Retour de pertinence indirect

- Analyse le comportement implicite de l'utilisateur (clics, temps de lecture)
- Ajuste la requête en fonction de ces signaux comportementaux
- Exemple :
 - Si les utilisateurs cliquent systématiquement sur des résultats parlant de "recettes" après une requête "tarte", le système peut automatiquement associer ces termes

Transformations de la requête

Utiliser le contexte de la requête, les informations de l'utilisateur

- Pour transformer la requête, on peut aussi s'aider des informations que l'utilisateur nous donne sans le savoir:
 - Sa localisation pour des requêtes liées à un lieu
Exemple:
Requete: restaurant
—> on va vouloir lui présenter les restaurants prêt d'où il se trouve
 - Son historique de requêtes pour :
 - Désambiguiser sa requête
 - Utiliser les clicks pour remonter les documents/domains préférés

Transformations de la requête

Utiliser le contexte de la requête, les informations de l'utilisateur

- Pour transformer la requête, on peut aussi s'aider des informations que l'utilisateur nous donne sans le savoir:
 - Mouvements de la souris : il y a une grande corrélation entre les mouvements de la souris et la fixation des yeux
 - Ce qui en fait un bon indicateur de **l'attention** que peut porter l'utilisateur sur un résultat
 - Mais ne fonctionne pas sur mobile
 - Le temps passé sur une page cliquée et si l'utilisateur revient sur la page de recherche, click à nouveau sur un résultat etc.
 - On calcule alors la **satisfaction** de l'utilisateur

Transformations de la requête

Points clés

- Les méthodes globales sont indépendantes du contexte et des résultats
- Les méthodes locales s'adaptent aux résultats et au comportement utilisateur
- La combinaison des deux approches permet une recherche plus précise et personnalisée

Transformations de la requête

Quels feedbacks par les IA? Interfaces “zero-click”

- Les IA comme Le Chat viennent chercher les réponses directement dans les SERP.

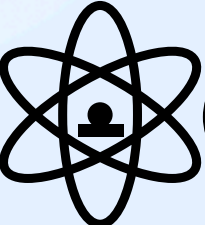
Conséquence : Les utilisateurs obtiennent une réponse sans cliquer sur les liens sources, ce qui réduit drastiquement les signaux de feedback traditionnels.

- Solution : Utiliser des LLMs pour simuler des jugements de pertinence

Transformations de la requête

Transformer la requête en vecteur

- Semantic search VS lexical search
- Recherche du plus proche voisin
 - Calculer des distances / similarités entre vecteurs

 Quelles sont les limites de cette approche ?

Transformations de la requête

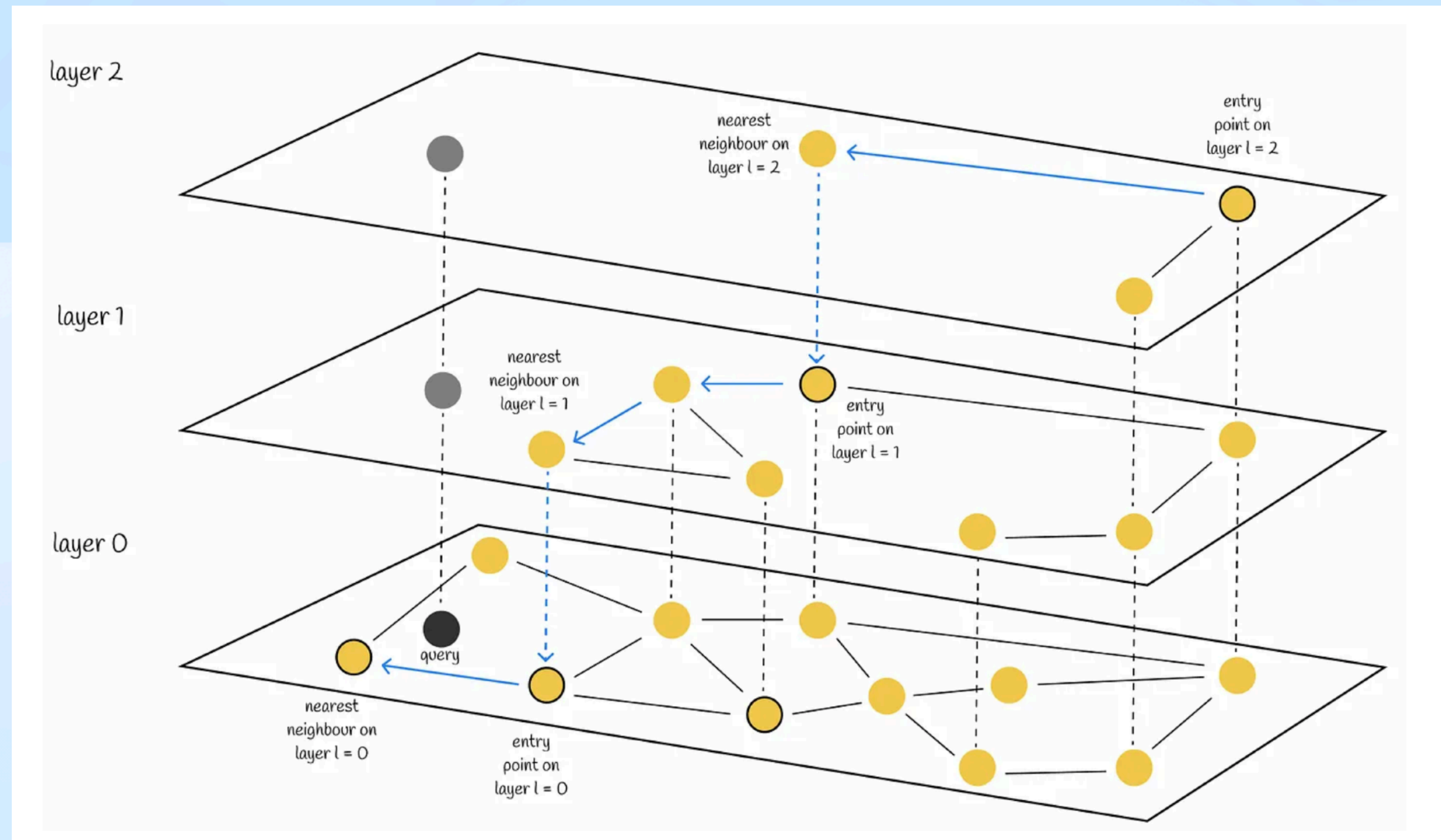
Transformer la requête en vecteur

- Approches ANN (Approximate Nearest Neighbors)
 - Approximer une recherche par similarité exacte, dans des scénarios où des solutions exactes sont computationnellement coûteuses ou impraticables
 - Precision < efficacité
 - On va calculer les distances entre les vecteurs des documents (indexées dans les index) et les organiser de manière à rester proches les uns des autres
 - à l'aide de graphes, d'arbres, de hachages

Transformations de la requête

Transformer la requête en vecteur

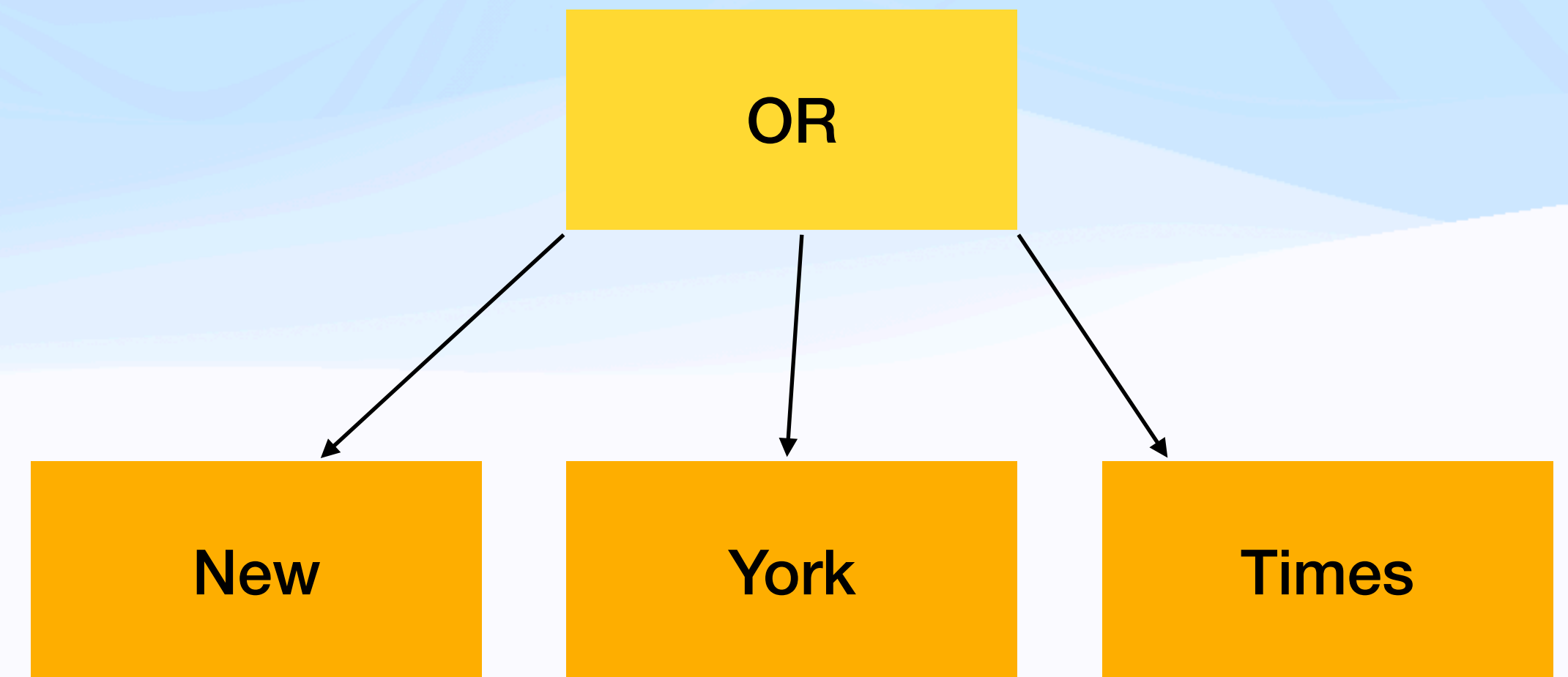
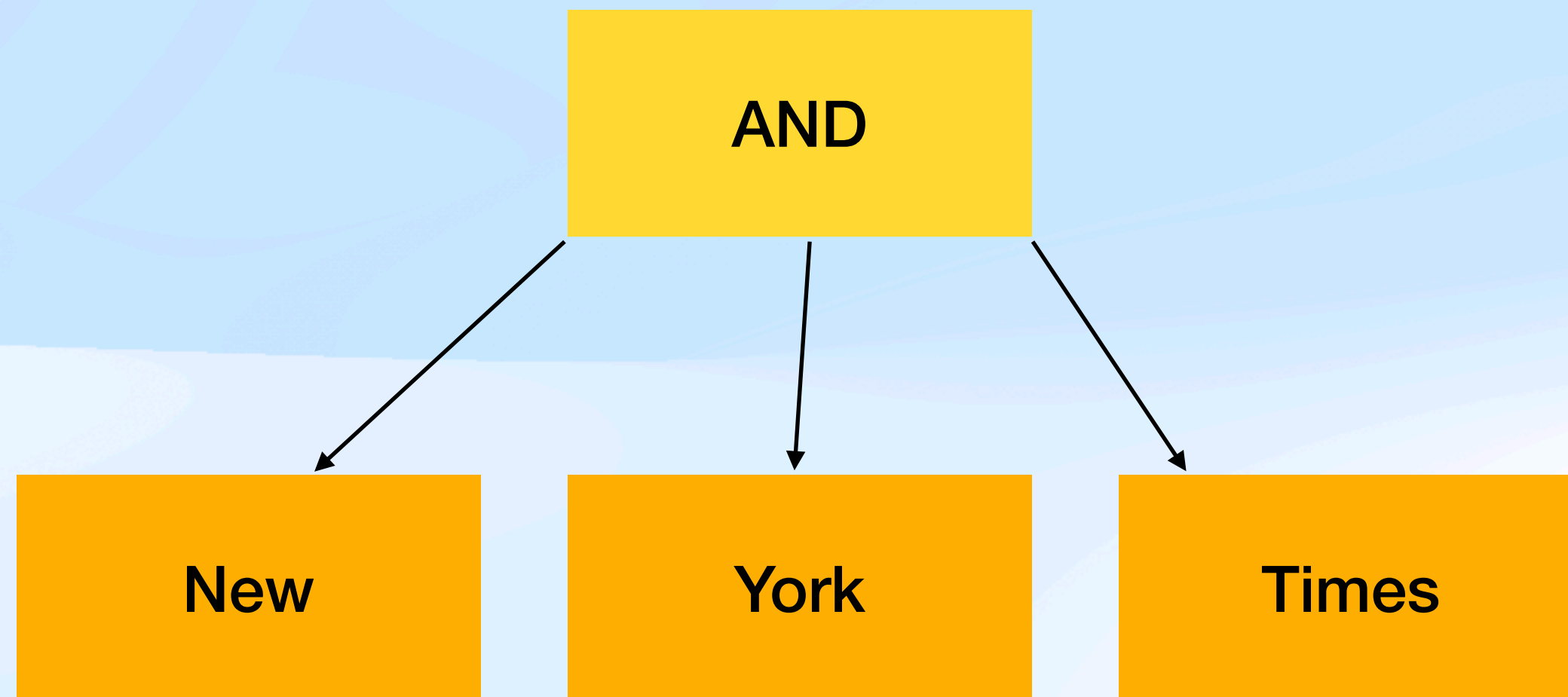
- Hierarchical Navigable Small Worlds (HNSW) -> $O(\log n)$



Représentation de la requête

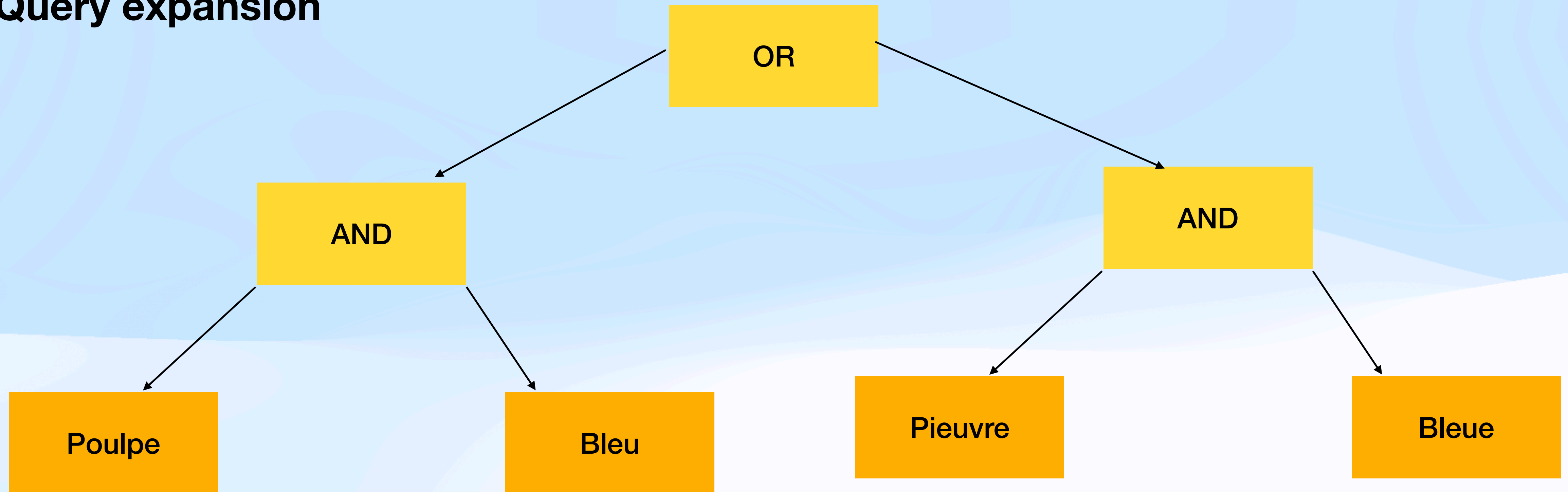
Représentation de la requête

Tel un arbre



Représentation de la requête

Query expansion



Partie 2

Traitement de la requête et ordonnancement des résultats

Ordonnancement des résultats et
évaluation

Ordonnancement des résultats

- On a maintenant:
 - Un index rempli de documents, chaque document est représenté par des champs (title, content, url, page rank etc.)
 - Un traitement de requêtes effectif avec de l'expansion de requête tels que le stemming qui nous permet de filtrer les documents qui pourraient correspondre à la requête, ou une transformation en vecteurs de documents et de la requête
- On veut:
 - Ordonner ces documents qui semblent pertinents pour répondre au mieux à l'utilisateur



Signaux

Ordonnancement des résultats (ranking)

Signaux

- Pour ordonner les résultats on va donner un score sur chaque document qui a survécu au filtre de la requête
- On va classer les signaux en plusieurs types en fonction de leur dépendance à la requête ou au document

Ordonnancement des résultats (ranking)

Signaux dépendant de la requête (et de l'utilisateur)

- Date et heure à laquelle a été lancée la requête
- Localisation de l'utilisateur
- Age de l'utilisateur
- Intention / topic
- Etc.

Ordonnancement des résultats (ranking)

Signaux dépendant du document

- Page rank et autres scores de popularité
- Spam / ham
- Topics
- Date de derniere mise a jour
- Langue du document
- Etc.

Ordonnancement des résultats (ranking)

Signaux dépendants du document et de la requête

- Ce sont des signaux qui vont calculer un score de similarité entre le texte de la requête (et de la requête étendue) avec le texte d'un ou plusieurs champs du document
- Similarité lexicale
 - Binaire
 - Taux de similarité
- Similarité vectorielle
 - Distance
 - Similarité

Ordonnancement des résultats (ranking)

Bm25(query, document.field, b, k)

$$\sum_i^n IDF(q_i) \frac{f(q_i, D) * (k1 + 1)}{f(q_i, D) + k1 * (1 - b + b * \frac{fieldLen}{avgFieldLen})}$$

- q_i -> token i de la requête
- $f(q_i, D)$ -> fréquence de q_i dans le champs du document
-> plus le token q_i apparait plus le score final sera élevé
- $fieldLen/avg(fieldLen)$ —> En d'autres termes, plus le document contient de tokens - du moins ceux qui ne correspondent pas à la requête - plus le score du document est faible.
Si un document de 300 pages mentionne ma requête une fois, il est moins probable qu'il ait un rapport avec elle qu'un court tweet qui la mentionne une fois.
- b contrôle l'impact du ratio
est un paramètre entre 0 et 1, défaut: 0.75
plus il est grand et plus $fieldLen/avg(fieldLen)$ a d'effet
- $K1$ limite à quel point un seul token dans la requête peut affecter le score —> stopwords, default: 1.2

Ordonnancement des résultats (ranking)

Bm25(query, document.field)

$$\sum_i^n IDF(q_i) \frac{f(q_i, D) * (k1 + 1)}{f(q_i, D) + k1 * (1 - b + b * \frac{fieldLen}{avgFieldLen})}$$

- IDF(qi) -> inverse document frequency
diminue le poids des termes qui apparaissent très fréquemment dans l'ensemble de documents et augmente le poids des termes qui apparaissent rarement

$$= \log \frac{N}{|\{d \in D : t \in d\}|}$$

Ordonnancement des résultats (ranking)

Bm25(query, document.field)

Exemple q = president Lincoln

docs -> 500 000

$K1 = 1.2$

$b = 0.75$

Frequency of “president”	Frequency of “lincoln”	BM25 score
15	25	20.66
15	1	12.74
15	0	5.00
1	25	18.2
0	25	15.66

Tâche de ranking

Ordonnancement des résultats (ranking)

Linear Ranking

- On va prendre plusieurs scores/fonctions de scoring et on va donner une importance plus ou moins forte sur chaque score:
- `ranking_function(q,d) ->`

```
Score <-  bm25(q, document.title) * 10  
          + page_rank * 20  
          + same_language(q, document)
```

- A quoi ça vous fait penser ?

Ordonnancement des résultats (ranking)

Learning to Rank

- Modèle d'apprentissage automatique qui apprend à prédire un score à partir d'un input $x = (q, d)$
Au lieu d'apprendre à minimiser une loss, il va apprendre à maximiser une métrique (choisie en amont, ex: recall, ndcg, map etc.)
- Decision tree, SVM, modèle multi couches
- Supervised, semi supervised, reinforcement learning
- 3 approches différentes
 - Pointwise
 - Pairwise
 - Listwise

Ordonnancement des résultats (ranking)

Learning to Rank - point wise

- Input: une paire (q,d) unique
- Loss: distance entre le score prédit et le vrai score
- Il faut le voir comme une tâche de régression

Ordonnancement des résultats (ranking)

Learning to Rank - pair wise

- Input: une paire (q , d_i , d_j)
- Loss: prédire lequel des deux documents est le plus pertinents
- Il faut le voir comme une tâche de classification binaire

Ordonnancement des résultats (ranking)

Learning to Rank - list wise

- Input: une paire $(q, [d_1, \dots, d_n])$
- Prend en compte tous les documents qui ont survécu au filtre
- A ce moment là, on intègre la métrique dans la loss
On va maximiser la métrique à l'entraînement

**Ordonner les résultats de
plusieurs moteurs de recherche**

Recherche fédérée

Federated Search

- La recherche fédérée permet d'interroger plusieurs sources de données hétérogènes (bases de données, APIs, moteurs de recherche, systèmes internes) en une seule requête utilisateur.
- Fonctionnement:
 - L'utilisateur saisit une requête unique
 - La requête est envoyée simultanément à plusieurs sources
 - Chaque source retourne ses résultats
 - Les résultats sont agrégés et présentés ensemble
- Limites:
 - Classement global complexe
 - Latence dépendante de la source la plus lente
 - Résultats parfois hétérogènes en qualité

Reclassement des résultats

Reranking

- Le reranking consiste à réordonner une liste de résultats déjà récupérés afin d'améliorer leur pertinence pour l'utilisateur.
- Fonctionnement:
 - Une première recherche génère une liste de résultats de plusieurs moteurs
 - Un modèle de reranking (règles ou IA) évalue chaque résultat
 - Les résultats sont reclassés selon un score de pertinence

Evaluation des résultats

Evaluation des résultats

- Plusieurs types d'évaluation de la pertinence d'un résultat
- Requete : url et non pas Requête : liste d'urls
- Pertinence de topic
Si les résultats ont le bon topic
VS
pertinence en fonction de l'utilisateur
Si les résultats sont assez "frais", s'il a la bonne langue etc.
- Binaire
pertinent ou non pertinent
VS
à plusieurs valeurs
plus ou moins pertinent

Evaluation des résultats

Evaluation binaire | Recall @ k

- Cette mesure indique combien de résultats pertinents réels ont été affichés parmi tous les résultats pertinents réels pour la requête
- $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$



$$\text{Recall@1} = 1/3 = 0.33$$



$$\text{Recall@3} = 2/(2+1) = 2/3 = 0.67$$

Quel est le recall@5 de ces deux modèles?



Evaluation des résultats

Evaluation binaire | Precision @ k

- Cette métrique quantifie le nombre d'éléments pertinents dans les résultats du top-K
- $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$



$$\text{Precision@1} = 1/1 = 1$$



$$\text{Precision@2} = 1/(1+1) = 1/2 = 0.5$$

quel est la Precision@5 de ces deux modèles?



Evaluation des résultats

Evaluation binaire | Average Precision

- Une métrique qui prend en compte l'ordre des documents retournés

$$AP = \frac{\sum_{k=1}^n (P(k) * rel(k))}{number\ of\ relevant\ items}$$

	1	2	3	4	5
Precision@K	1	1/2	2/3	2/4	3/5

$$AP = \frac{(1 + 2/3 + 3/5)}{3} = 0.7555$$

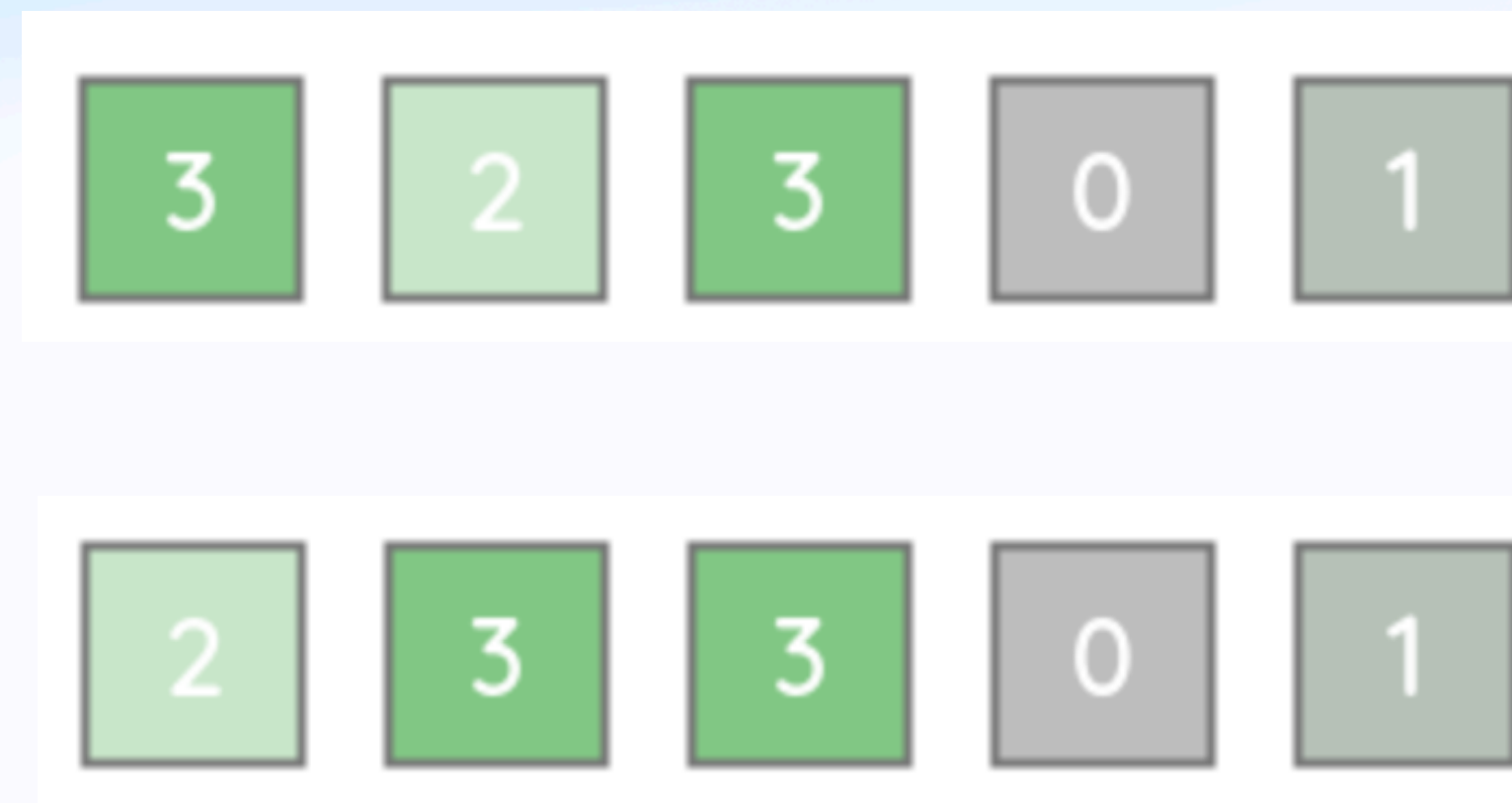
quel est l' Average Precision de ces deux modèles?

	1/1	2/2	3/3	3/4	3/5
Model A	1	2	3	4	5
Model B	1	2	3	4	5
	0/1	0/2	1/3	2/4	3/5

Evaluation des résultats

Evaluation multivaleurs | Cumulative Gain (CG)

- Si on a un score de pertinence sur nos documents, il y a des métriques qui prennent en compte ces scores:
- Cumulative Gain -> somme des scores
 - CG@3 ?



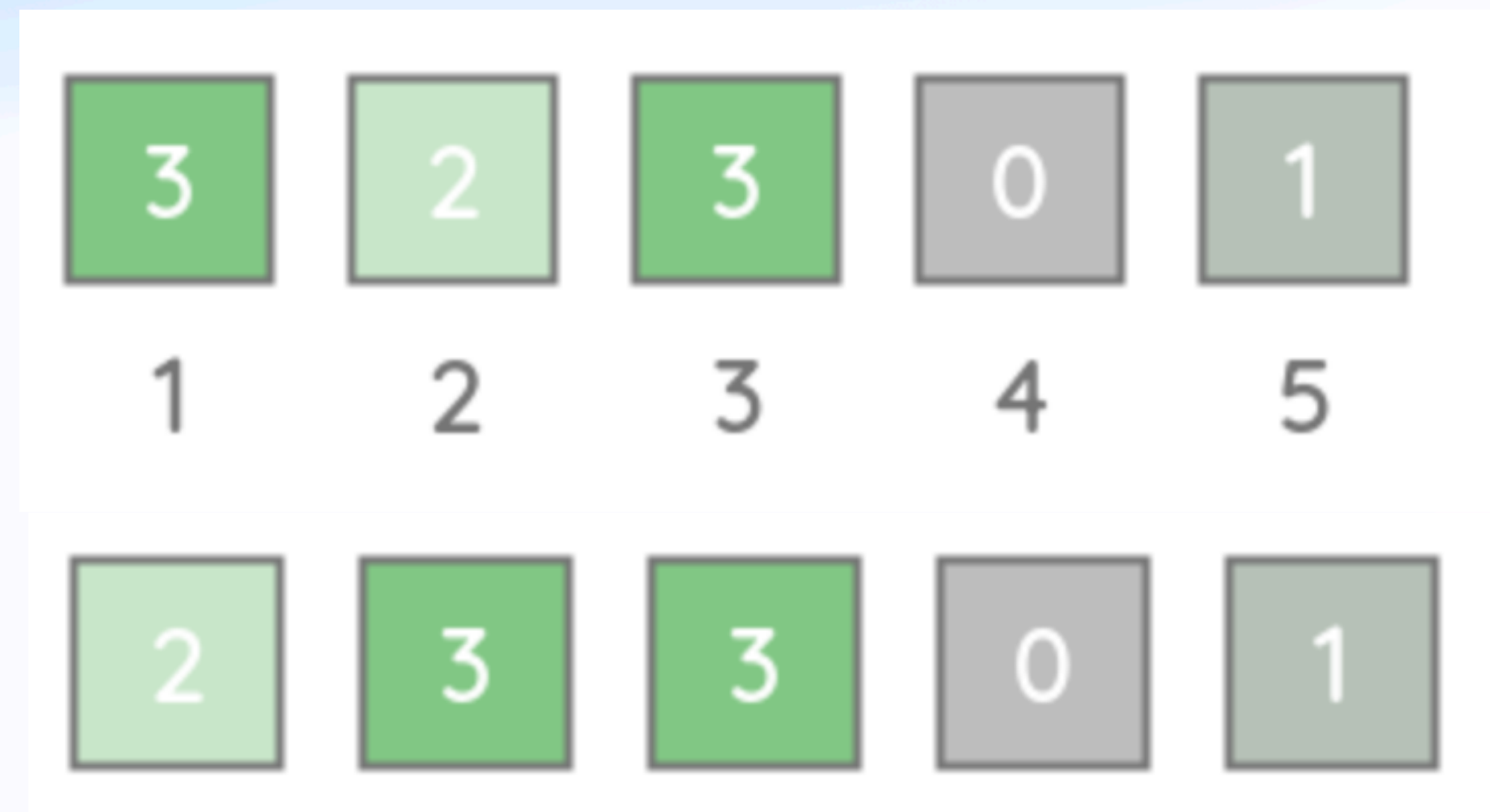
Evaluation des résultats

Evaluation multivaleurs | Discounted Cumulative Gain (DCG)

- Nous avons donc besoin d'un moyen de pénaliser les scores en fonction de leur position. Le DCG introduit une fonction de pénalité basée sur le logarithme pour réduire le score de pertinence à chaque position.
- Discounted Cumulative Gain

$$DCG@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}$$

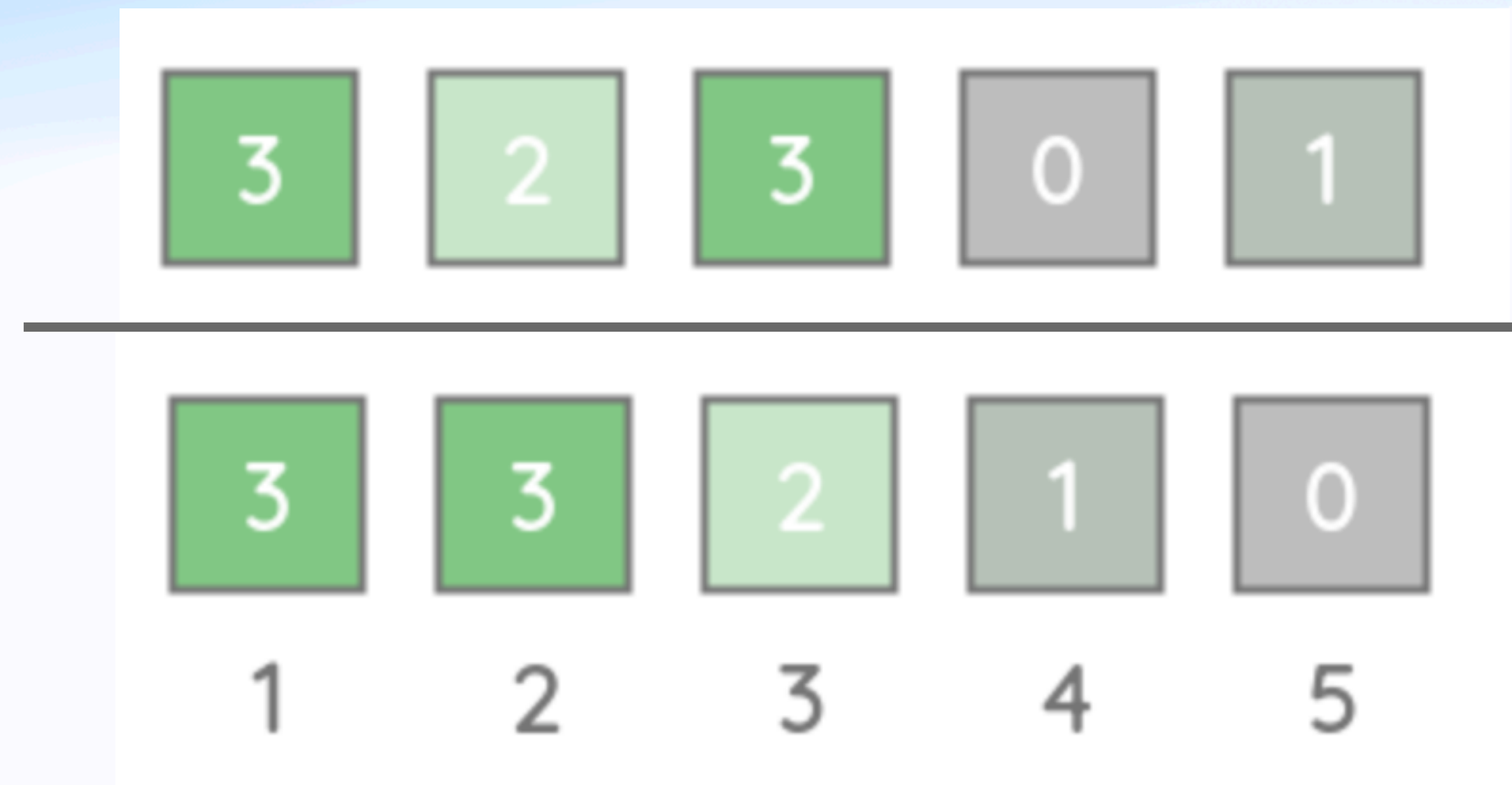
- $DCG@3 = 3 + 1.26 + 1.5 = 5.76$
- $DCG@3 = 2 + 1.89 + 1.5 = 5.39$



Evaluation des résultats

Evaluation multivaleurs | Normalised Discounted Cumulative Gain (NDCG)

- Pour permettre une comparaison du DCG entre les requêtes avec des résultats de taille différente
 - $NDCG@k = DCG@k / IDC@k$
 - Avec $IDCG = \text{ideal dcg}$



Partie 2

Traitement de la requête et ordonnancement des résultats

LLMs et les moteurs de recherche

Chat GPT

- Generative AI (GPT3.5 - Generative Pre-trained Transformer 3.5)
- Processus d'entraînement
 - Pré-entraînement :
 - Dataset: énormément de documents provenant du web
 - Tâche: prédiction du prochain token.
 - Fine-tuning : Après le pré-entraînement, le modèle est affiné sur un ensemble de données plus restreint avec l'aide d'évaluateurs humains -> affinent le comportement du modèle sur la base de critères souhaités, tels que l'absence de résultats biaisés ou de contenu inapproprié.

LLMs

- Limites :
 - Manque de compréhension du monde réel
 - Sensibilité à la formulation des entrées
 - Risque de résultats biaisés
 - Incapacité à vérifier les informations
- Les défis connus des LLM sont les suivants :
 - Présenter de **fausses informations** lorsqu'il n'y a pas de réponse.
 - Présenter des **informations périmées** ou génériques alors que l'utilisateur attend une réponse spécifique et actuelle.
 - Créer une réponse à partir de sources ne faisant pas autorité.
 - La création de réponses inexactes en raison d'une confusion terminologique, lorsque différentes sources de formation utilisent la même terminologie pour parler de choses différentes.
- “I apologize, but as an AI language model, I do not have real-time data or access to current news. My knowledge was last updated in September 2021, and I cannot provide you with the latest developments on the topic beyond that point.”

RAG

Retrieval-Augmented Generative

- Combinaison modèles génératifs + approches basées sur la recherche d'informations
- Les systèmes traditionnels de recherche d'informations s'appuient souvent sur des bases de données ou des documents préexistants pour récupérer les informations pertinentes, alors que les modèles génératifs excellent dans la génération de textes de type humain sur la base des données d'entrée.
- Les RAG font le lien entre ces deux paradigmes pour créer un système plus polyvalent et plus efficace.

RAG

Retrieval-Augmented Generative

- Modèle génératif (par exemple, GPT) : permet au modèle de générer des réponses cohérentes et adaptées au contexte.
- Module de recherche : contribue à garantir que les réponses générées sont non seulement cohérentes, mais aussi fondées sur des informations factuelles et pertinentes.

RAG

Retrieval-Augmented Generative

- Pré-entraînement : même pré-entraînement que celui d'un GPT
- Finetuning fin avec le module de recherche
 - Personnalisation (domaines / tâches spécifiques)
 - Adaptabilité aux changements dans les index
- Applications :
 - Intégrés dans les moteurs de recherche
 - Chatbot

RAG

Retrieval-Augmented Generative

- Autres avantages
 - Simple à mettre en place
 - Renforcement de la confiance des utilisateurs
 - Plus de contrôle de la part des développeurs

RAG

Tools

- LangChain
- Pinecone
- LLamaIndex

Indexation web

Et après?

- Un ranking pour toutes les requêtes ou plusieurs rankings spécialisés ?
Pour ou contre?
- Représentations multimodales, que fait-on des images ?
- Comment présenter les résultats?

Industrie

- Quel plateforme devrais-je utiliser ?
 - Apache Solr
Solr est le projet de moteur de recherche le plus ancien
Grande communauté
 - Elastic Search
d'abord concentré sur la facilité d'utilisation
Axé sur les applications analytiques
La plus grande des communautés
 - Vespa.ai
Vespa a l'avantage de prendre en charge nativement les fonctions de recherche “modernes” (LTR)
La communauté Vespa est encore petite